CHAPTER 2 • GIT & GITHUB MASTERY ~15 Min

Student

Can you explain what this chapter is about?

Claude • AI Assistant

Chapter 1 got AskMyNotes onto GitHub with disciplined commits. **Chapter 2 turns solo Git into professional collaboration** - the workflow that every real engineering team runs, every day.

Section 2.1 covers branches: what they really are (movable pointers, not folder copies), the universal feature-branch workflow, and the three real-world strategies you must be able to name in an interview - **trunk-based development** (main-only with feature flags, standard at big continuous-delivery shops), **GitHub Flow** (short feature branches + PRs, the sensible mid-team default), and **GitFlow** (formal ceremony for versioned releases). **Section 2.2** goes deep on **rebasing and cherry-picking** - one of the two high-emphasis skills of the chapter.

The mechanics of rebase vs merge, the **golden rule** ('never rebase commits others have pulled'), **interactive rebase** for cleaning up messy local commits before a PR, cherry-picking to backport fixes, and safe force-pushing with `--force-with-lease` instead of raw `--force` .

Student

Can you break down the next concept?

Claude • AI Assistant

Section 2.3 is the second high-emphasis subsection - **merge conflict resolution**. The most anxiety-inducing Git skill for beginners, treated with extra depth.

Anatomy of the `<<<<<<< HEAD` / `=======` / `>>>>>>>` markers, the **five-step resolution ritual** (breathe -> status -> edit -> add -> continue), the difference between merge and rebase conflict contexts, editor tooling that makes it easy, and the **abort escape hatch** - `git merge --abort` and `git rebase --abort` cleanly restore your pre-conflict state, so nothing is ever truly stuck.

Section 2.4 covers Pull Request etiquette: the size rule (PRs under 400 lines get real review; giant PRs are rubber-stamped), the What/Why/How/Test-plan description template, how to give and receive review feedback constructively, draft PRs for work-in-progress, and branch protection settings that enforce quality. **Section 2.5** demystifies **Git internals** - the four object types (blob, tree, commit, tag) stored content-addressed in `.git/objects/`, how HEAD and refs work, and the **reflog** as the safety net that keeps every commit for 90 days even after resets, rebases, or deleted branches.

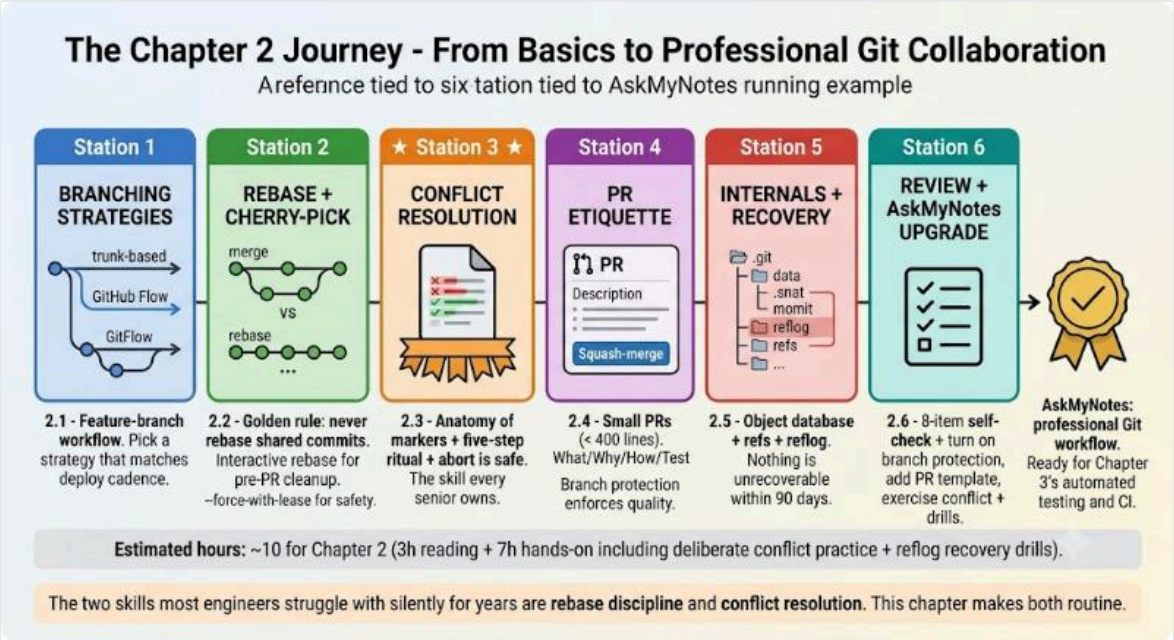
Nothing is unrecoverable once you know `git reflog` + `git reset --hard HEAD@{N}` or `git branch rescue <hash>`.

Student

Can you break down the next concept?

Claude • AI Assistant

By the end of Chapter 2 your AskMyNotes repo has: branch protection on `main`, a PR template, a clean history from squash-merges, and you personally have muscle memory for conflict resolution and reflog-based recovery. That combination is professional-grade Git fluency - the baseline for **Chapter 3**, which adds automated testing and continuous integration on top.



The Chapter 2 Journey - From Basics to Professional Git Collaboration

Chapter 2 at a Glance - Professional Git Collaboration

Subsection	Title	Core Question Answered	You Can Now…
2.1	Branching Strategies	How do real teams organize branches?	Run the feature-branch workflow fluently and pick the right strategy (trunk-based / GitHub Flow / GitFlow) for a team profile
2.2	Rebasing and Cherry-Picking (high-emphasis)	When to rebase vs merge, and how to do it safely?	Rebase feature branches for clean history, use interactive rebase to clean up before PR, cherry-pick specific commits, and force-push safely with --force-with-lease
2.3	Merge Conflict Resolution (high-emphasis)	How do I handle conflicts without panicking?	Read conflict markers correctly, execute the five-step resolution ritual, use editor visual merge tools, and know abort is always safe
2.4	PR Etiquette and Code Review	How do I get code reviewed and merged well?	Write PRs under 400 lines with structured descriptions, give and receive review feedback constructively, enable branch protection, and pick the right merge strategy
2.5	Git Internals and Recovery	What's inside .git, and how do I recover from mistakes?	Name the four object types, read HEAD and refs, use the reflog to recover from resets/rebases/deleted branches, and never fear a Git accident again
2.6	Chapter Review + Summary	What do I own now?	Answer the 8-item self-check and complete the AskMyNotes professional-workflow exercise

bash — Chapter 2 preview - the professional Git workflow, condensed

```
# ---- Feature branch, professional flow ----
git checkout main
git pull --rebase
git checkout -b feature/rerank
# ... work + commit + commit + commit ...
```

```
# ---- Keep current with main via rebase (linear history) ----
git fetch origin
git rebase origin/main             # replays your commits on latest main
# conflicts? -> five-step ritual: status/edit/add/continue

# ---- Clean up messy commits before opening PR ----
git rebase -i HEAD~5               # squash / drop / reword

# ---- Push and open PR ----
git push -u origin feature/rerank
gh pr create --base main --title "Add cross-encoder re-ranker" \
  --body-file .github/pull_request_template.md

# ---- After review + approval + green CI, merge via GitHub UI ----
# Choose 'Squash and merge' -> one clean commit on main
# GitHub auto-deletes the source branch

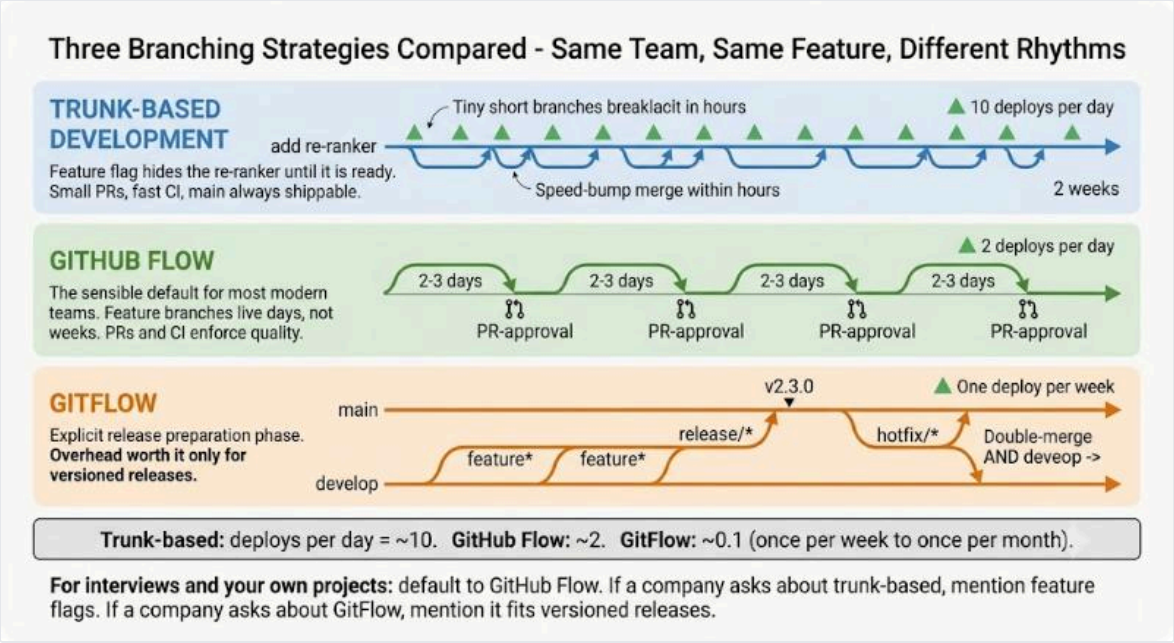
# ---- Sync local and clean up ----
git checkout main
git pull
git branch -d feature/rerank
git remote prune origin

# ---- WHEN THINGS GO WRONG - the recovery moves ----
git merge --abort           # bail out mid-merge
git rebase --abort           # bail out mid-rebase
git reflog                   # every HEAD move for 90 days
git reset --hard HEAD@{2}    # jump back to a reflog entry
git branch rescue <hash>     # revive a 'lost' commit as a branch

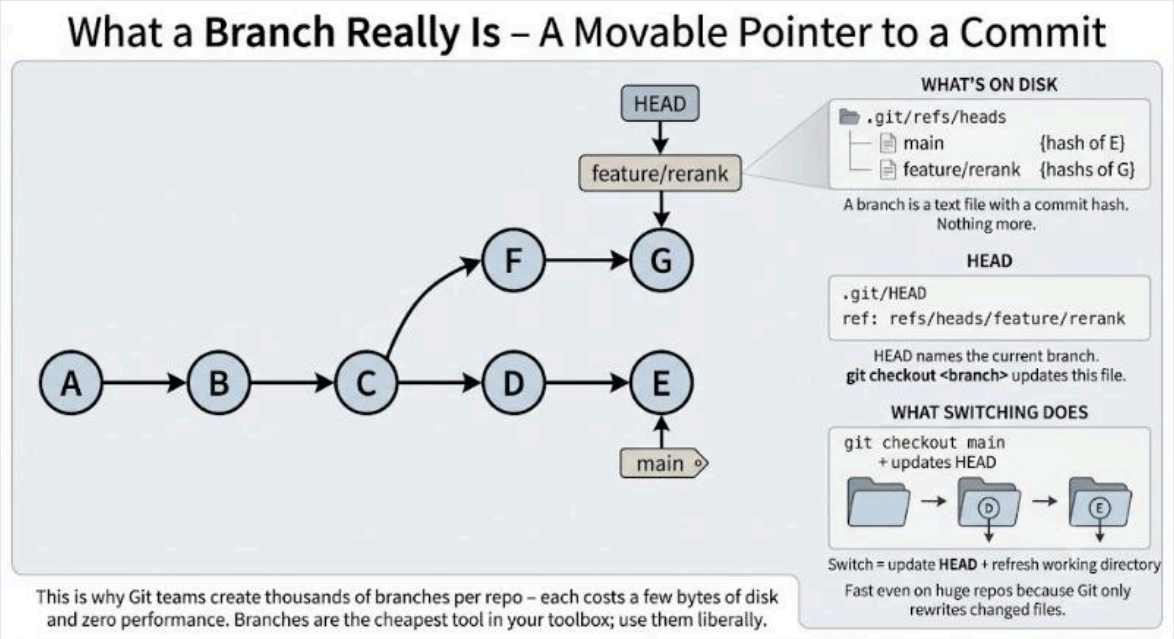
# ---- Force-push safely after rebase ----
git push --force-with-lease origin feature/rerank
# NEVER just --force (blast radius: teammate's commits)
```

BRANCHING STRATEGIES - FEATURE BRANCHES, GITFLOW, TRUNK-BASED

Claude · AI Assistant



Three Branching Strategies Compared - Same Team, Same Feature, Different Rhythms



What a Branch Really Is - A Movable Pointer to a Commit

bash — The feature-branch workflow, end to end

```bash
# ---- Start fresh from latest main ----
git checkout main
git pull --rebase

# ---- Create + switch to a feature branch ----
git checkout -b feature/rerank-retrieval
# Modern equivalent:
# git switch -c feature/rerank-retrieval
```

```
# ---- Work on the branch: commit as many times as you like ----
code retrieval.py tests/test_rerank.py
git add retrieval.py tests/test_rerank.py
git commit -m "Add cross-encoder re-ranker on top-K retrieval"

code README.md
git add README.md
git commit -m "Document re-ranker in README"

# ---- Keep in sync with main while working (avoid drift) ----
git fetch origin
git rebase origin/main       # replay your commits on top of latest main
# (If a conflict pops up, we handle it in 2.3.)

# ---- Push and open a PR ----
git push -u origin feature/rerank-retrieval
# (Open the PR in the GitHub UI. GitHub will suggest a link right after
push.)

# ---- After PR is approved, CI is green, and merge is done on GitHub ----
git checkout main
git pull                                    # bring the merged commit local
git branch -d feature/rerank-retrieval     # delete LOCAL branch
git push origin --delete feature/rerank-retrieval  # delete REMOTE branch

# ---- Inspect all branches ----
git branch -a
#   * main
#     remotes/origin/HEAD -> origin/main
#     remotes/origin/main
# Clean state - ready for the next feature branch.

# ---- Bonus: prune remote-tracking refs for branches deleted on GitHub ----
git remote prune origin
# or set once:
git config --global fetch.prune true
```

bash — GitFlow-style commands (for reference - most teams no longer use this)

```
# GitFlow uses multiple long-lived branches. Real teams typically use
# the 'git-flow' helper tool that wraps these commands, but the underlying
# operations are plain git. Shown here for interview vocabulary.

# ---- Setup: main + develop as long-lived branches ----
```

```
git checkout main
git checkout -b develop
git push -u origin develop

# ---- Start a feature (branches off develop, not main) ----
git checkout develop
git pull
git checkout -b feature/rerank
# ... work, commit, push ...
# When ready, PR from feature/rerank -> develop

# ---- Start a release (branches off develop) ----
git checkout develop
git checkout -b release/1.2.0
# Only bug fixes and release-prep commits allowed here
# When ready:
git checkout main
git merge --no-ff release/1.2.0
git tag -a v1.2.0 -m "Release 1.2.0"
git checkout develop
git merge --no-ff release/1.2.0            # merge back into develop too!
git branch -d release/1.2.0

# ---- Hotfix a production bug (branches off main) ----
git checkout main
git checkout -b hotfix/1.2.1-null-query
# ... fix, commit ...
git checkout main
git merge --no-ff hotfix/1.2.1-null-query
git tag -a v1.2.1 -m "Hotfix 1.2.1"
git checkout develop
git merge --no-ff hotfix/1.2.1-null-query  # merge into develop too!
git branch -d hotfix/1.2.1-null-query

# Notice: every release and hotfix has to be merged into BOTH main AND
develop.
# This is the ceremony that made teams tire of GitFlow. Trunk-based avoids it
entirely.
```

Student

Chapter 1 got AskMyNotes onto GitHub with commits. But in real teams, nobody just commits straight to main - there are branches, PRs, review, and merge strategies. Where

do I even start?

Claude • AI Assistant

Branches are Git's superpower. They are cheap - a branch in Git is just a movable pointer to a commit, not a copy of files - so real teams create them liberally: one branch per feature, per bug fix, per experiment.

The interesting question is not 'should we branch?' but 'what's our strategy?' - meaning what long-lived branches exist, what short-lived branches are allowed, and how they merge back. This subsection walks through the three real-world strategies (trunk-based, GitHub Flow, GitFlow) and gives you the vocabulary to answer 'what's your branching model?' in any team interview.

WHAT A BRANCH ACTUALLY IS

Student

Under the hood, what IS a branch?

Claude • AI Assistant

CONCEPT CARD

Not what beginners imagine. A branch is **not** a copy of your files. It's a **tiny text file** inside `.git/refs/heads/` containing one line - the commit hash the branch currently points to:

bash — Code Example

```
cat .git/refs/heads/main
# a3f2c1b7e4d99c1a8b3e5f6d7e8f9a0b1c2d3e4f
```

That's it. When you `git commit`, Git creates a new commit and advances the branch pointer to it.

When you `git checkout feature/x`, Git updates the pointer file called HEAD to say 'currently on feature/x' and re-materializes the working directory to match that branch's commit. Creating a branch is nearly free (a few bytes on disk); switching between branches is fast even on giant repositories.

This is why Git teams create thousands of branches per repo without hesitation - branches are the cheapest thing in Git.

FEATURE BRANCHES - THE UNIVERSAL BUILDING BLOCK

Student

What's the most common pattern?

Claude • AI Assistant

CONCEPT CARD

The **feature branch workflow** is the foundation of every modern strategy. The rule: **never commit directly to main; do all work on short-lived branches; merge back via Pull Request after review and CI**.

bash — Code Example

```bash
# You want to add a cross-encoder re-ranker to AskMyNotes retrieval
git checkout main
git pull                                    # start from latest main
git checkout -b feature/rerank-retrieval    # create + switch

# Edit, add, commit as many times as needed
# ... work ...
git add retrieval.py tests/test_rerank.py
git commit -m "Add cross-encoder re-ranker on top-K retrieval"

# Push branch and open a Pull Request
git push -u origin feature/rerank-retrieval

# After PR approval and merge on GitHub, clean up
git checkout main
git pull
git branch -d feature/rerank-retrieval           # delete local branch
git push origin --delete feature/rerank-retrieval # delete remote branch
```

Branch naming conventions vary by team - common patterns: `feature/<short-name>`, `bugfix/<issue-number>-<slug>`, `hotfix/<slug>`, `experiment/<slug>`. Consistency inside a team matters more than which convention you pick.

STRATEGY 1 - TRUNK-BASED DEVELOPMENT

Student

What's the simplest strategy?

Claude • AI Assistant

CONCEPT CARD

Trunk-based development keeps one long-lived branch (main / trunk) and forbids anything else from lasting more than a day. Feature branches are ultra-short-lived: create in the morning, PR at lunch, merge by evening.

If a feature is bigger than a day, either **split it** into smaller mergeable pieces, or ship it behind a **feature flag** (hidden by default, enabled for testers, then enabled for everyone when ready).

Advantages: near-zero merge conflicts, main is always shippable, CI runs on tiny diffs, small PRs are easy to review, no long-lived divergence.

Requirements: strong test suite + fast CI (< 10 min), feature flag capability, discipline to keep changes small.

Where it dominates: Google, Facebook, Netflix, most modern web companies. This is the state of the art for continuous delivery.

STRATEGY 2 - GITHUB FLOW

Student

And GitHub Flow?

Claude • AI Assistant

CONCEPT CARD

GitHub Flow is trunk-based with slightly longer branches (1-5 days), formalized around Pull Requests:

1. Create a feature branch off `main` .
2. Commit incrementally on the branch.
3. Open a Pull Request early - even before the code is done - and use it for discussion.
4. CI runs on every push to the PR.
5. Reviewers approve; the PR merges into main; main is deployed.

This is what most small-to-mid teams (2-50 people) actually run. If you join a modern web-app or SaaS company as a fresher, this is very likely the workflow.

Compared to trunk-based, GitHub Flow allows slightly longer branches; compared to GitFlow, it has none of the ceremony of `develop` / `release` / `hotfix` branches.

STRATEGY 3 - GITFLOW (CLASSIC)

Student

What about GitFlow?

Claude • AI Assistant

CONCEPT CARD

GitFlow (proposed by Vincent Driessen in 2010) uses multiple long-lived branches:

- **`main`** (or `master`) - only production-released code; each commit is tagged with a version.
- **`develop`** - integration branch where features land first.
- *`feature/**` - short-lived branches off develop, merged back into develop`.
- *`release/**` - branched from develop when preparing a release; only bug fixes go here; merged to both main and develop` when shipped.
- *`hotfix/**` - branched from main for urgent production fixes; merged to both main and develop`.

Advantages: clean separation of 'what's in production' vs 'what's next'; explicit release preparation phase.

Disadvantages: complex, lots of ceremony, easy to mis-manage, merge-heavy. Modern web-app teams have largely moved away from GitFlow because continuous delivery + feature flags achieve the same goals with less overhead.

Where it still dominates: teams with formal versioned releases - mobile app teams (App Store review cycles), library authors, embedded systems, some enterprise software. Even the author of GitFlow now recommends trunk-based for continuous-delivery web apps.

CHOOSING THE RIGHT STRATEGY

Student

How does a team decide?

Claude • AI Assistant

CONCEPT CARD

Ask three questions:

1. **Do we deploy multiple times a day, or ship in versioned releases?** Continuous delivery -> trunk-based or GitHub Flow. Versioned releases -> GitFlow or environment branches.
2. **How strong is our test automation?** Weak tests -> longer branches make sense (more manual verification per merge). Strong tests -> short branches are better (fewer conflicts).
3. **How large is the team?** Small teams tolerate any strategy. Large teams (100+) require the discipline of trunk-based to avoid branch chaos.

For your projects and most interviews, **default to GitHub Flow** - it's the sensible middle ground and understood everywhere.

APPLIED TO ASKMYNOTES

Student

How would we branch AskMyNotes?

Claude • AI Assistant

EXAMPLE CARD

Practically, for a solo student project:

- **main** is your only long-lived branch. Always deployable (or at least always in a runnable state).
- **Feature branches** for anything more than a one-line change: `feature/rerank`, `feature/pdf-tables`, `bugfix/chunker-off-by-one`.
- **Open a PR** even to yourself. It shows the diff cleanly, runs CI, and gives you a reviewable summary.
- **Delete branches after merge** - both local and remote.

This is essentially GitHub Flow with a team of one. When you join a real team, the workflow will feel familiar.

ANALOGY

Branching strategies are like **document collaboration modes** in Google Docs. **Trunk-based** is like everyone editing the live doc simultaneously with tight coordination - only works if the doc is short and changes are small.

GitHub Flow is like each person copies the doc, edits their copy for a few hours, then merges back via 'suggesting mode' review - works well for most teams. **GitFlow** is like maintaining an 'in progress' doc, a 'this quarter's release' doc, a 'shipped' doc, and formal promotion between them - overhead worth it only when the audience expects distinct versioned releases (a printed magazine, not a live blog).

REAL WORLD APPLICATION

A payments startup ran GitFlow for two years and shipped every three weeks. They switched to trunk-based with feature flags and started shipping 20+ times a day.

Deployment risk actually went DOWN because each deploy contained a tiny change (easy to diagnose if something broke). The GitFlow release-preparation ceremony had been theater; the small-batch discipline of trunk-based was the real quality lever.

WARNING

Never commit directly to `main` on a team repo - even a one-character typo fix. It bypasses review, bypasses CI, and (on protected branches) will be rejected by GitHub.

Enforce this by turning on **branch protection** in GitHub Settings: require PR, require passing CI, require at least one reviewer.

COMMON PITFALL / MISTAKE

Long-lived feature branches. Once a branch lives more than a week, main has drifted so far that merging back is painful.

If a feature is legitimately big, either (a) merge incremental slices behind a feature flag, or (b) rebase your branch onto main frequently to keep it current. Every extra day a branch lives is compounded merge risk.

EXAM TIP CARD

Interview question: 'Describe your team's branching strategy.' If you have none: 'For my projects I use GitHub Flow - short-lived feature branches off main, PR with CI, merge and delete.' Then name the alternatives you know (trunk-based, GitFlow) and when each fits. Naming trade-offs sounds senior; parroting one strategy as universally best sounds junior.

PRACTICE QUIZ

Your team ships to production 30 times per day and has a 3-minute CI suite. Which branching strategy fits best?

A) GitFlow with formal release branches

B) Environment branches (dev/staging/prod as long-lived)

C) Trunk-based with feature flags for incomplete work

D) One long-lived branch per developer

▼ Reveal Correct Answer

Answer: C. High deploy frequency + fast CI is exactly the environment trunk-based was designed for. GitFlow (A) adds ceremony that fights against continuous delivery. Environment branches (B) create drift. Per-developer branches (D) is chaos.

EXERCISE

On AskMyNotes: (a) create a branch `feature/log-retrieval-latency`, (b) add a simple `time.time()` around the retrieval call and log the latency, (c) commit with a proper 50/72 message, (d) push and open a PR against main (even if you're the only reviewer), (e) merge, then delete the branch locally and remotely. Confirm `git branch -a` shows only main afterward.

AI DEEP DIVE PROMPT

Ask an AI: '*Compare trunk-based development, GitHub Flow, and GitFlow. For each: describe the branching model, the typical team profile that uses it, its main advantage, and its main downside. Then recommend one for a 5-person team building a SaaS product.*'

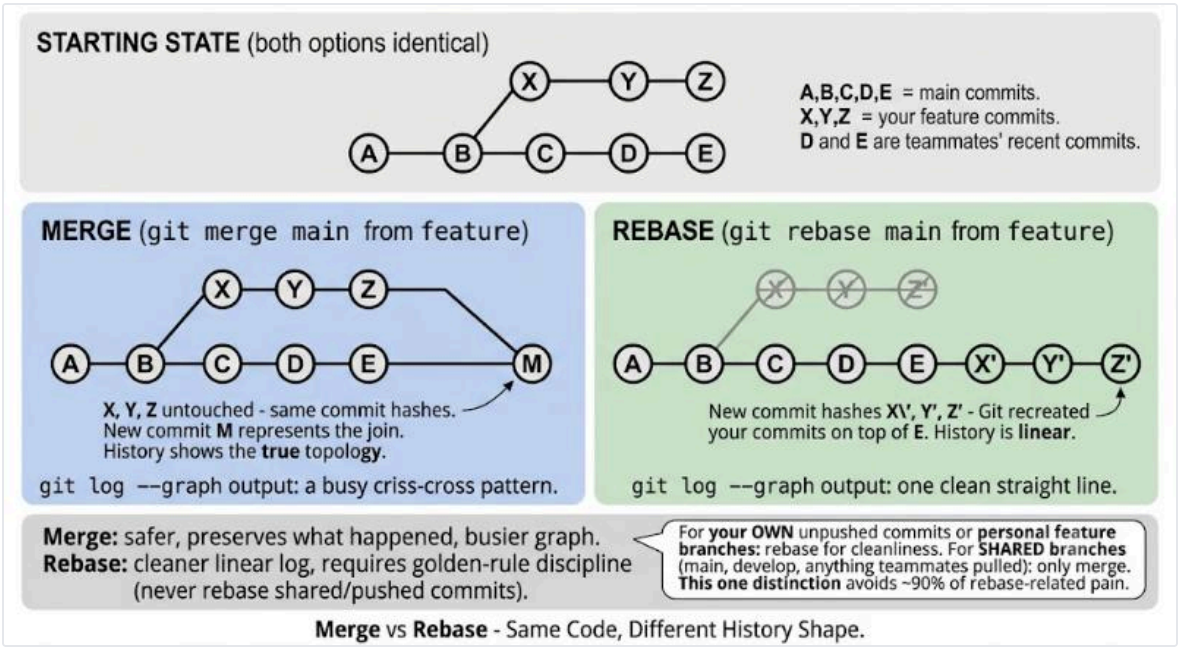
SUMMARY CARD

A **branch** in Git is just a movable pointer to a commit - cheap to create, fast to switch. The universal building block is the **feature branch workflow**: never commit directly to main; branch, commit, push, PR, review, merge, delete.

Three real-world strategies: **Trunk-based** (main-only, ultra-short branches, feature flags - the modern default at big continuous-delivery shops), **GitHub Flow** (main + short feature branches with PRs - the sensible small/mid team default), **GitFlow** (main + develop + release + hotfix - ceremony worth it only for versioned releases). Default to GitHub Flow for your projects; know the alternatives for interviews.

REBASING AND CHERRY-PICKING - THE HIGH-EMPHASIS SKILL

Claude • AI Assistant



Merge vs Rebase - Same Code, Different History Shape



Interactive Rebase - Turning Messy Local History Into a Clean PR

bash — The rebase workflow - keeping a feature branch current and tidy

```
# ---- Scenario: feature/rerank has drifted from main. Rebase to catch up. --
--
git checkout feature/rerank
git fetch origin
git rebase origin/main
# If conflicts appear:
#   Edit conflicting files (see 2.3 for detail)
```

```
#   git add <resolved-file>
#   git rebase --continue
# If you get lost:
#   git rebase --abort   (returns to exactly where you were)

# You've already pushed the branch before. Now local and remote diverge.
# Use --force-with-lease (NOT plain --force) to overwrite safely:
git push --force-with-lease origin feature/rerank

# ---- Interactive rebase: clean up the last 5 commits before opening PR ----
git checkout feature/rerank
git rebase -i HEAD~5
# Editor opens. Change 'pick' to 'squash' / 'fixup' / 'reword' / 'drop' as
needed.
# Save and close - Git executes the plan.

# ---- Cherry-pick a specific commit from main onto a release branch ----
# Scenario: hotfix a3f2c1 landed on main. release/1.9 needs the same fix.
git checkout release/1.9
git pull
git cherry-pick a3f2c1
# Conflict? Edit files, git add, git cherry-pick --continue.
# Abort? git cherry-pick --abort.
git push

# ---- Range cherry-pick: bring several commits at once ----
git cherry-pick a3f2c1..b7e4d9        # excludes a3f2c1, includes b7e4d9
git cherry-pick a3f2c1^..b7e4d9       # includes both

# ---- Setting pull to rebase globally for cleaner history ----
git config --global pull.rebase true  # 'git pull' now behaves as 'git pull -
-rebase'

# ---- Recover from a botched rebase using the reflog ----
git reflog                             # shows every HEAD position, even
rewritten history
# find the pre-rebase entry, then:
git reset --hard HEAD@{5}              # jump back to that state
# (git internals + reflog covered in detail in 2.5)
```

Student

Everyone tells me rebasing is dangerous. But experienced engineers use it constantly. Why is this so controversial - and when should I actually use it?

Claude • AI Assistant

Because **rebasing rewrites history**. Merge preserves what actually happened; rebase pretends things happened in a different order to keep the log linear.

Both produce identical final code - the trade-off is purely about history shape and safety. There is exactly one hard rule (the 'golden rule of rebasing' - never rebase commits that others have pulled) and a lot of nuance around it.

This subsection is deliberately deeper than most because rebasing is where most engineers get burned early and where the greatest history-quality gains come from.

THE PROBLEM REBASE SOLVES

Student

Show me why rebase exists.

Claude • AI Assistant

CONCEPT CARD

You started a `feature/rerank` branch off `main` last Tuesday. Since then, teammates have merged 7 commits into main.

Your branch has 3 commits of your own. If you `git merge main` into your branch, Git creates a **merge commit** joining the two histories.

Now your branch history has a fork-and-join shape.

Multiply this by every feature branch in a busy repo and `git log --graph` becomes an unreadable spaghetti of merge commits, most of which say things like 'Merge branch main into feature/rerank'. Signal-to-noise ratio drops.

Rebase alternative: instead of merging main INTO your branch, you replay your 3 commits ON TOP OF the current main tip. History looks as if you started your work today, after the 7 teammate commits landed. Linear, clean, one strand.

Trade-off: your 3 commits are **not the same commits anymore** - they have new parent commits, so new commit hashes. You have rewritten your own local history to make the shared repository look tidy.

THE MECHANICAL EXPLANATION

Student

Walk through what actually happens.

Claude • AI Assistant

EXAMPLE CARD

Setup:

```
code — Code Example
main:     A --- B --- C --- D --- E   (D, E are teammates' recent commits)
               \
feature:        X --- Y --- Z         (your 3 commits started from B)
```

Option 1 - MERGE main into feature:

```
bash — Code Example
git checkout feature
git merge main
```

Result:

```
code — Code Example
main:     A --- B --- C --- D --- E
               \               \
feature:        X --- Y --- Z --- M   (M = merge commit)
```

Your commits (X, Y, Z) are untouched. A new merge commit M ties the branches together.

Option 2 - REBASE feature onto main:

```
bash — Code Example
git checkout feature
git rebase main
```

Result:

code — Code Example

```
main:     A --- B --- C --- D --- E
                                   \
feature:                            X' --- Y' --- Z'
```

Your commits are gone; new commits X', Y', Z' have replaced them - same code changes and messages, but different parent (E instead of B), so different hashes. History is now linear.

Both options produce identical files when Z or Z' is checked out. The difference is entirely in history shape.

WHEN TO REBASE, WHEN TO MERGE

Student

What's the rule?

Claude • AI Assistant

CONCEPT CARD

Two guidelines that cover 95% of cases:

Rule 1 - The Golden Rule of Rebasing: **never rebase commits that other people have pulled**. Rewriting shared history breaks anyone who has a copy of the original commits - their next pull will diverge painfully.

Rebase only YOUR local, unpushed commits (or commits on a feature branch nobody else is working on).

Rule 2 - Style guideline: rebase your feature branch onto main frequently to keep it current, but merge feature branches into main via a Pull Request. Some teams prefer 'squash merge' (compress all feature-branch commits into one commit on main) for the cleanest main history.

Common practice at most modern teams:

- Work on a feature branch.
- Rebase your branch onto main every day or two (linear, always current).
- Open a PR when done.
- Merge the PR via GitHub with 'Squash and merge' or 'Rebase and merge'.
- Delete the branch.

Result: **main has a beautiful linear history, one commit per feature**. That's the artifact you'll thank yourself for six months later.

INTERACTIVE REBASE - EDITING HISTORY

Student

What about `-i` ?

Claude • AI Assistant

CONCEPT CARD

`git rebase -i` opens an editor listing the commits you're about to rebase, with actions you can apply per commit:

code — Code Example

```
pick a3f2c1 Add cross-encoder re-ranker
pick b7e4d9 Fix typo in re-ranker docstring
pick c9a1b3 Add tests for re-ranker
pick d2f5e8 WIP debugging
pick e8b2c4 Actually fix the re-ranker bug

# Commands:
# p, pick   = use commit
# r, reword = use commit, but edit the commit message
# e, edit   = use commit, but stop for amending
# s, squash = use commit, but meld into previous commit
# f, fixup  = like squash, but discard this commit's message
# d, drop   = remove commit
```

Common uses:

- **Squash** several messy WIP commits into one clean commit.
- **Reword** a commit message you regret.
- **Reorder** commits into a logical narrative.
- **Drop** an experimental commit you don't want.

bash — Code Example

```
# Clean up the last 5 commits before opening a PR
git rebase -i HEAD~5
```

```
# In the editor, change:
#   pick a3f2c1 Add cross-encoder re-ranker
#   pick b7e4d9 Fix typo in re-ranker docstring
#   pick c9a1b3 Add tests for re-ranker
#   pick d2f5e8 WIP debugging
#   pick e8b2c4 Actually fix the re-ranker bug
# to:
#   pick a3f2c1 Add cross-encoder re-ranker
#   fixup b7e4d9 Fix typo in re-ranker docstring
#   pick c9a1b3 Add tests for re-ranker
#   drop d2f5e8 WIP debugging
#   fixup e8b2c4 Actually fix the re-ranker bug
# Save, close. Git squashes and reorders as instructed.
```

The result: 5 messy commits become 2 clean ones on your local branch. **Then** you push (with `--force-with-lease` if you'd already pushed the messy version).

GIT PULL --REBASE

Student

What about pulling with rebase?

Claude • AI Assistant

CONCEPT CARD

`git pull` = `git fetch` + `git merge`. This can create annoying tiny merge commits when both you and a teammate committed to main during a sync gap.

`git pull --rebase` = `git fetch` + `git rebase` - replays your local commits on top of the freshly-fetched remote commits, giving a linear log.

Many teams make this the default:

bash — Code Example

```
git config --global pull.rebase true
```

Once set, plain `git pull` behaves as `git pull --rebase` everywhere. Safer default for feature branches.

CHERRY-PICKING

Student

And cherry-pick?

Claude • AI Assistant

CONCEPT CARD

`git cherry-pick <commit>` copies a single commit from another branch onto the current branch. Common uses:

Backporting a hotfix: a bug was fixed on `main` (commit `a3f2c1`); the same fix is needed on a still-supported `release/1.9` branch.

bash — Code Example

```
git checkout release/1.9
git cherry-pick a3f2c1
git push
```

Rescuing a commit from a wrong branch: you accidentally committed a fix to `main` when you meant to commit to `feature/x`. Cherry-pick to the right branch, then revert on the wrong one.

Range cherry-pick: `git cherry-pick A..B` picks all commits after A up to and including B.

As with rebase, cherry-pick creates a NEW commit (new hash) even though the changes are identical. Use judiciously.

FORCE PUSHING SAFELY

Student

What if I've already pushed and then I rebase?

Claude • AI Assistant

⚠ WARNING

After a rebase your local branch and the remote branch have diverged (same code, different history). A normal `git push` will be rejected because the histories don't fast-forward.

`git push --force` overwrites the remote history with yours. Dangerous - if a teammate pushed to the branch since your last fetch, their commits vanish.

`git push --force-with-lease` is the safer variant. It force-pushes ONLY if the remote branch is still at the commit your local repo thought it was - i.e., nobody else pushed in the meantime.

If someone did, the push is rejected and you know to fetch and reconcile before overwriting.

bash — Code Example

```
git push --force-with-lease origin feature/rerank
```

Always prefer `--force-with-lease` over `--force`. And never force-push to `main` or any shared long-lived branch without explicit team coordination.

ANALOGY

Merge is like **taping two documents together with a visible seam**; rebase is like **retyping one document as if it had always been part of the other**. The final content is the same; the artifact reads differently.

For a personal notebook, either is fine. For a shared textbook that others have already copied, retyping the shared parts (rebasing pushed commits) breaks everyone's copy - use the tape instead.

REAL WORLD APPLICATION

A backend engineer at a fintech had a habit of `git push --force` on their feature branches to keep them tidy. One day, without noticing, they force-pushed to a branch a teammate had also pushed to.

The teammate's four commits vanished from the remote. The teammate still had them locally, so nothing was permanently lost - a `git push --force-with-lease` on the teammate's side restored them - but the incident triggered a team-wide switch to `--force-with-lease` as the default.

Small habit, big blast-radius avoidance.

WARNING

Never rebase commits that others have based work on. If you rebase and force-push a branch that teammates have pulled, their next fetch shows two versions of every commit (old originals still in their reflog, new post-rebase versions on origin) and they'll spend hours untangling it.

The golden rule exists for a reason.

COMMON PITFALL / MISTAKE

Running `git rebase main` from `main` instead of from your feature branch. This tries to replay main's history onto itself - nonsensical, often a no-op, sometimes destructive. Always check `git branch` before rebasing to confirm you're on the branch you mean to modify.

[Common Mistake #2] Panicking mid-rebase when a conflict appears and running `git reset --hard` or deleting the repo. **Don't**.

`git rebase --abort` cleanly cancels the rebase and returns you to exactly where you started. `git rebase --continue` (after resolving conflicts) proceeds.

There is no rebase state so broken that these two commands can't recover it.

EXAM TIP CARD

Star interview question: **'What is the golden rule of rebasing?'** Perfect answer: 'Never rebase commits that others have pulled. Rebase rewrites history - if others have pulled the original commits, your rewritten versions diverge from what they have, and their next fetch will be a mess.

Rebase is for local, unpushed commits or your own feature branches that no one else is working on.'

PRACTICE QUIZ

You're on `feature/rerank` (2 commits ahead of main). Main has moved forward 5 commits. You want your feature branch to appear as if you started work AFTER those 5 commits landed, so history stays linear. What do you run?

A) git checkout main && git merge feature/rerank

B) git checkout feature/rerank && git merge main

C) git checkout feature/rerank && git rebase main

D) git checkout feature/rerank && git reset --hard main

▼ Reveal Correct Answer

Answer: C. Rebasing feature onto main replays your 2 commits on top of main's tip, giving linear history. A merges feature into main - opposite direction. B creates a merge commit on feature. D erases your commits entirely.

EXERCISE

On AskMyNotes: (a) create branch `feature/logging`, make 3 tiny commits with messy messages ('wip', 'more wip', 'actually working'), (b) run `git rebase -i HEAD~3` and squash

them into ONE clean commit with a good message, (c) verify `git log --oneline -3` shows the collapsed history, (d) if you already pushed, `git push --force-with-lease`.

AI DEEP DIVE PROMPT

Ask an AI: '*Explain git merge vs git rebase using an ASCII diagram of before-and-after commit graphs. Then describe when to use each, including the golden rule of rebasing and when force-with-lease is safe.*'

SUMMARY CARD

Merge joins two branches with a merge commit; **rebase** replays your commits on top of another branch, giving linear history but new commit hashes. Both produce identical code.

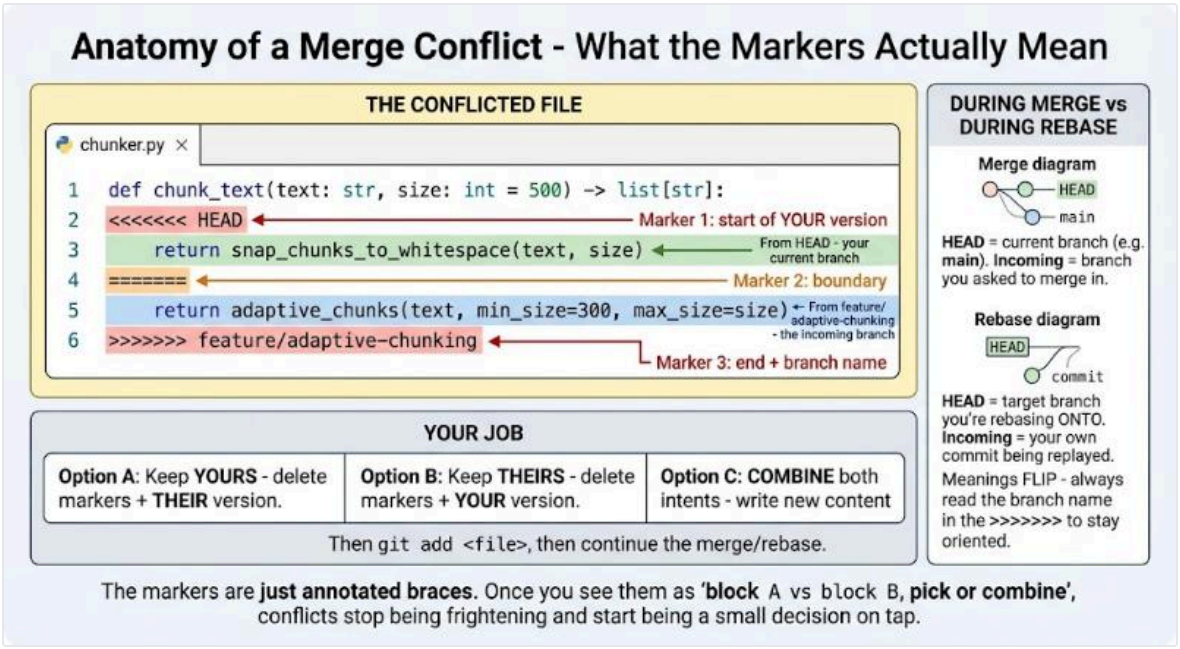
Golden rule of rebasing: never rebase commits others have pulled. Practical workflow: rebase your feature branch onto main frequently for cleanliness, merge to main via PR (squash-merge for maximum tidiness).

Interactive rebase (`git rebase -i`) lets you squash, reorder, drop, and reword local commits before opening a PR. **Cherry-pick** copies a specific commit to another branch (backports, rescues).

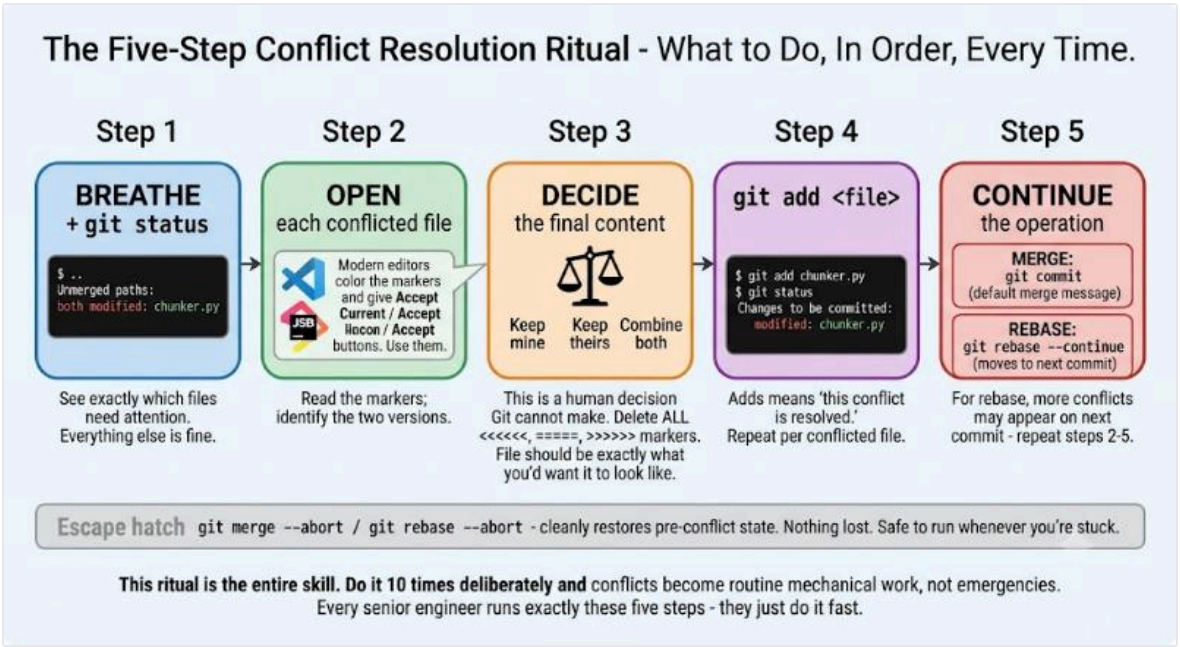
Always prefer `--force-with-lease` over `--force`. `git rebase --abort` cleanly cancels; `git rebase --continue` proceeds after conflict resolution.

MERGE CONFLICT RESOLUTION - THE SKILL EVERY ENGINEER FEARS AND EVERY SENIOR OWNS

Claude • AI Assistant



Anatomy of a Merge Conflict - What the Markers Actually Mean



The Five-Step Conflict Resolution Ritual - What to Do, In Order, Every Time

bash — The full conflict-resolution session - merge and rebase variants

```bash
# ============================================================
# SCENARIO 1: MERGE conflict (git merge or git pull)
# ============================================================
git checkout main
git merge feature/whitespace-snap
# Auto-merging chunker.py
# CONFLICT (content): Merge conflict in chunker.py
# Automatic merge failed; fix conflicts and then commit the result.
```

```
# Step 1: See what needs resolving
git status

# Step 2-3: Open chunker.py, edit to remove markers + settle on final content
code chunker.py    # or your editor of choice
# Delete <<<<<<< HEAD, =======, >>>>>>> lines. Write the final code.

# Step 4: Mark resolved
git add chunker.py
git status         # should show 'All conflicts fixed but you are still
merging.'

# Step 5: Complete the merge
git commit          # opens editor with default 'Merge branch...' message
# or:
git commit -m "Merge feature/whitespace-snap; combine snap + adaptive"

# ============================================================
# SCENARIO 2: REBASE conflict (git rebase or git pull --rebase)
# ============================================================
git checkout feature/adaptive-chunking
git rebase main
# CONFLICT (content): Merge conflict in chunker.py
# Could not apply a3f2c1... Add adaptive chunk sizing

# Step 1: See what needs resolving
git status

# Step 2-3: Edit the file to remove markers, decide final content
code chunker.py

# Step 4: Mark resolved
git add chunker.py

# Step 5: Continue the rebase (repeats for each conflicting commit)
git rebase --continue

# ============================================================
# ESCAPE HATCH: You're lost. Abort. Try again from scratch.
# ============================================================
git merge --abort            # during a merge
git rebase --abort           # during a rebase
git cherry-pick --abort      # during a cherry-pick

# ============================================================
```

```
# WHOLESALE ACCEPT (when combining is not needed)
# ============================================================
git checkout --ours chunker.py     # keep YOUR version entirely
git checkout --theirs chunker.py   # keep THEIR version entirely
git add chunker.py

# ============================================================
# CONFIGURE A VISUAL MERGE TOOL (optional but recommended)
# ============================================================
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd \
  'code --wait --merge $REMOTE $LOCAL $BASE $MERGED'
# During a conflict:
git mergetool                      # opens VS Code 3-way merge view

# ============================================================
# SAFETY CHECK: No marker leftovers before committing
# ============================================================
grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> ' .
# Any hits = marker was not fully removed. Fix before committing.
# Chapter 4 shows how to enforce this via pre-commit hook.
```

Student

Every time Git says 'CONFLICT (content):' my heart stops. I usually panic-copy the file somewhere, delete the branch, and start over. How do professionals actually handle this?

Claude • AI Assistant

Calmly. **Merge conflicts are Git's way of asking a human to make a decision** - specifically, when two branches edited the same lines of the same file in different ways, Git cannot tell which change to keep.

It stops, marks the file with special conflict markers, and hands the pen to you. That's it.

There is nothing broken; nothing is lost; you have not damaged the repository. The panic reaction most students have is entirely learned - and this subsection unlearns it.

By the end you will treat conflicts as routine mechanical work, not emergencies.

WHY CONFLICTS HAPPEN

Student

Under what conditions does Git bail?

Claude • AI Assistant

CONCEPT CARD

Git can auto-merge any change where the two sides did not touch the same lines. It fails only when both branches edited the same region of the same file - specifically when there's no unambiguous way to combine them. Common triggers:

- **You and a teammate both edited the same function** on your respective branches.
- **You and a teammate both added imports at the top of a file** (both edits landed on the same line).
- **A file was renamed on one branch and modified on the other**.
- **A file was deleted on one branch and modified on the other**.
- **A rebase replays commits over a moving target** - each commit might conflict independently.

Git flagging a conflict doesn't mean anyone did anything wrong. It means the tool cannot decide safely and needs your judgment.

ANATOMY OF A CONFLICT MARKER

Student

What do those `<<<<<<<` lines mean?

Claude • AI Assistant

CONCEPT CARD

When Git can't auto-merge, it writes both versions into the file with marker lines around them. Here's a realistic conflict in AskMyNotes' `chunker.py` :

python — Code Example

```
def chunk_text(text: str, size: int = 500) -> list[str]:
<<<<<<< HEAD
    # Fixed-size chunks with 50-char overlap
    return [text[i:i+size] for i in range(0, len(text), size - 50)]
=======
```

```
    # Snap to whitespace boundaries to avoid splitting words
    return snap_chunks_to_whitespace(text, size)
>>>>>>> feature/whitespace-snap
```

Reading the markers:

- **`<<<<<<< HEAD`** - the version on YOUR currently-checked-out branch (usually main or the branch you were on when the merge started).
- Content between `<<<<<<< HEAD` and `=======` - YOUR version.
- **`=======`** - the boundary between the two sides.
- Content between `=======` and `>>>>>>>` - THEIR version (from the branch being merged in).
- **`>>>>>>> feature/whitespace-snap`** - names the branch the other version came from.

During a **rebase**, the meanings flip: `HEAD` is the branch you're rebasing ONTO (usually main), and the other side is your own commit being replayed. Read the branch name in the `>>>>>>>` marker to keep it straight.

THE FIVE-STEP RESOLUTION RITUAL

Student

Give me the exact process, step by step.

Claude • AI Assistant

CONCEPT CARD

Every conflict, whether merge or rebase, resolves the same way:

Step 1 - Breathe. Run `git status`. See which files have `both modified` under 'Unmerged paths'. Those are the ones needing attention. Everything else is fine.

Step 2 - Open each conflicted file in your editor. Modern editors (VS Code, JetBrains, Sublime) highlight the conflict markers in color and provide 'Accept Incoming' / 'Accept Current' / 'Accept Both' buttons directly in the gutter. Use them.

On a plain text editor, edit manually.

Step 3 - Decide what the final content should be. Options: keep your version, keep theirs, combine both, or write something new that captures both intents.

Delete the conflict markers (`<<<<<<<` , `=======` , `>>>>>>>`). The final content should be exactly what you'd want the file to look like.

Step 4 - `git add <file>` for each resolved file. This tells Git 'this conflict is done'. Verify with `git status` - the file should move from 'Unmerged' to 'Changes to be committed'.

Step 5 - Complete the operation. For a merge: `git commit` (Git provides a default 'Merge branch X' message; keep or edit it). For a rebase: `git rebase --continue` (which commits your resolution and moves to the next commit to replay).

That's the entire skill. Repeat per conflicted file; repeat per commit if it's a rebase.

A WORKED EXAMPLE

Student

Walk through the AskMyNotes chunker conflict end to end.

Claude • AI Assistant

EXAMPLE CARD

Setup: you're on main; your teammate merged `feature/whitespace-snap`; you have your own local branch `feature/adaptive-chunking` that also touched `chunker.py`.

bash — Code Example

```
# You're on your branch; want to catch up with main
git checkout feature/adaptive-chunking
git fetch origin
git rebase origin/main
```

Git output:

code — Code Example

```
Auto-merging chunker.py
CONFLICT (content): Merge conflict in chunker.py
error: could not apply a3f2c1... Add adaptive chunk sizing
```

bash — Code Example

```
# Step 1 - assess
git status
# rebase in progress; onto b7e4d9
# Unmerged paths:
```

```
#   (use "git add <file>..." to mark resolution)
#   both modified:   chunker.py
```

Step 2-3 - open `chunker.py` , see:

python — Code Example

```python
def chunk_text(text: str, size: int = 500) -> list[str]:
<<<<<<< HEAD                                          # main (with
teammate's snap logic)
    return snap_chunks_to_whitespace(text, size)
=======
    return adaptive_chunks(text, min_size=300, max_size=size)  # your work
>>>>>>> a3f2c1... Add adaptive chunk sizing
```

You want BOTH ideas combined - adaptive sizing that also snaps to whitespace. Edit to:

python — Code Example

```python
def chunk_text(text: str, size: int = 500) -> list[str]:
    raw = adaptive_chunks(text, min_size=300, max_size=size)
    return [snap_chunks_to_whitespace(c, size) for c in raw]
```

bash — Code Example

```bash
# Step 4 - mark resolved
git add chunker.py
git status
# rebase in progress; onto b7e4d9
# Changes to be committed:
#   modified:   chunker.py

# Step 5 - proceed with the rebase
git rebase --continue
# ... rebase continues; if more commits have conflicts, repeat per commit
...
```

When it finishes: `git log --oneline` shows your commits replayed cleanly on top of main. Done. Push with `--force-with-lease` if the branch was already published.

EDITOR TOOLING MAKES THIS EASY

Student

Isn't there a tool for this?

Claude • AI Assistant

CONCEPT CARD

Yes - use it.

VS Code: opens conflicted files with color-coded blocks and 'Accept Current Change' / 'Accept Incoming Change' / 'Accept Both Changes' / 'Compare Changes' buttons in the gutter. Zero manual marker deletion.

JetBrains IDEs (PyCharm, IntelliJ): `VCS -> Resolve Conflicts` opens a 3-way merge view showing left (yours), right (theirs), center (result you're building). Click arrows to accept individual changes; save when done.

Command-line 3-way merge tool: `git mergetool` opens whichever tool you've configured (`git config --global merge.tool vscode` sets VS Code; other options: meld, kdiff3, vimdiff).

Beginners often struggle with text-only conflict resolution and then discover their editor already handles it visually. Enable it early.

THE ABORT ESCAPE HATCH

Student

What if I get completely lost?

Claude • AI Assistant

CONCEPT CARD

Absolutely nothing is broken - abort and start over:

bash — Code Example

```
git merge --abort      # during a merge
git rebase --abort     # during a rebase
git cherry-pick --abort  # during a cherry-pick
```

Each of these cleanly restores the working directory, index, and HEAD to exactly the state you were in before the operation started. You've lost nothing except the time spent on the

failed attempt.

Say it out loud: **abort is safe**. Once you internalize this, conflicts stop being scary - the worst case is 'abort, think, try again'.

SEMANTIC VS TEXTUAL CONFLICTS

Student

One more thing - are all conflicts caught by Git?

Claude • AI Assistant

⚠ **WARNING**

No. Git detects **textual conflicts** - same lines edited on both sides. It does NOT detect **semantic conflicts** - where the code merges cleanly at the text level but breaks at the logic level.

Example: teammate renamed the function `chunk_text` to `split_text` on their branch. You added a call to `chunk_text` on your branch.

Git merges both changes cleanly (different lines). The result compiles-until-runtime, then blows up with `NameError: name 'chunk_text' is not defined`.

Defense: **always run the test suite after merging**. Even if Git reports no conflicts, `pytest` will surface most semantic breakages. Chapter 3 gets us building the test suites; Chapter 4 hooks them into CI so this check is automatic.

PREVENTING CONFLICTS BEFORE THEY HAPPEN

Student

How do teams minimize conflicts in the first place?

Claude • AI Assistant

CONCEPT CARD

Six habits:

1. **Small, short-lived branches** - the less your branch diverges from main, the fewer conflicts.

2. **Rebase frequently onto main** - catch conflicts one at a time instead of all at once at the end.
3. **Pull before you start work** - and pull again before you push. Half of all conflicts come from stale local branches.
4. **Communicate about big refactors** - if you're about to rename `chunker.py` to `text_splitter.py`, tell the team before you push.
5. **Keep files small and focused** - two people editing a 3000-line file is a conflict factory; the same two people editing separate 200-line files rarely conflict.
6. **Format code consistently** - use a shared formatter (`black` for Python) so nobody's edits are cluttered by whitespace-only changes.

ANALOGY

A merge conflict is like **two co-authors independently editing the same paragraph of a shared document**. Word can't tell whose version wins because both changed the same words.

Track Changes surfaces the disagreement and asks 'which do you want?' - or you rewrite the paragraph combining both ideas. Git is doing the same thing.

There's no bug, no corruption - just a decision that only a human can make.

REAL WORLD APPLICATION

A team lead runs a weekly workshop for new hires titled 'Conflict Resolution in 20 Minutes'. She creates deliberately conflicting PRs, walks through the marker anatomy, has each new hire resolve one live in VS Code, then aborts and re-does it via the command line.

The single biggest jump in Git confidence her team measures happens in this session. The skill isn't hard - it's **exposure that's missing**.

Same reason we're spending an extra-deep subsection on it.

WARNING

Do NOT commit files that still contain `<<<<<<<`, `=======`, or `>>>>>>>` markers. Some tools (like pre-commit hooks with the `check-merge-conflict` plugin, covered in Chapter 4) will block this automatically; without one, a file with markers still in it will break on next execution.

Grep for markers before committing: `grep -rn '<<<<<<<' .`

COMMON PITFALL / MISTAKE

Panicking and running `git reset --hard` or deleting the whole repo mid-conflict. This can lose work. Use `git merge --abort` or `git rebase --abort` instead - they cleanly

restore the pre-conflict state. Save `git reset --hard` for after you understand it (Chapter 2.5).

EXAM TIP CARD

Beloved live-coding exercise: **the interviewer creates a small conflict and asks you to resolve it**. What they're looking for: (1) `git status` first (situational awareness), (2) open the file and read the markers correctly, (3) edit to a sensible resolution, (4) `git add` the file, (5) `git merge --continue` or `git rebase --continue` .

Bonus points for saying 'and I'd run the tests after to catch semantic conflicts.' Don't need speed - need calm competence.

PRACTICE QUIZ

You run `git pull` and see:

▼ Reveal Correct Answer

code — Code Example

```
Auto-merging chunker.py
CONFLICT (content): Merge conflict in chunker.py
Automatic merge failed; fix conflicts and then commit the result.
```

What's your FIRST move?

A) git reset --hard HEAD to undo everything

B) Delete chunker.py and pull again

C) git status to see what needs resolving

D) Force-push to overwrite the remote

Answer: C. Always start with git status - it tells you exactly which files are unmerged. A can lose work. B loses your local edits. D is destructive and wrong for a conflict. C leads to the calm five-step ritual.

EXERCISE

Deliberately create a conflict in AskMyNotes to practice: (a) on main, edit line 1 of README to 'Line from main'; commit. (b) create branch `test/conflict` , checkout, edit the same line to 'Line from branch'; commit.

(c) `git checkout main` then `git merge test/conflict` -> conflict! (d) Open README.md, see the markers, resolve to 'Line from both branches', `git add` , `git`

`commit .`

(e) `git branch -d test/conflict` . Do this three times until it feels boring.

AI DEEP DIVE PROMPT

Ask an AI: '*Simulate a git merge conflict scenario for me: give me two hypothetical branches, show me the exact content of the file with conflict markers, and walk me through the five steps to resolve it. Then repeat the scenario in a rebase context so I see the difference.*'

SUMMARY CARD

Merge conflicts happen when two branches edit the same lines of the same file and Git can't safely auto-decide. Not a bug, not a corruption - a decision handed to you.

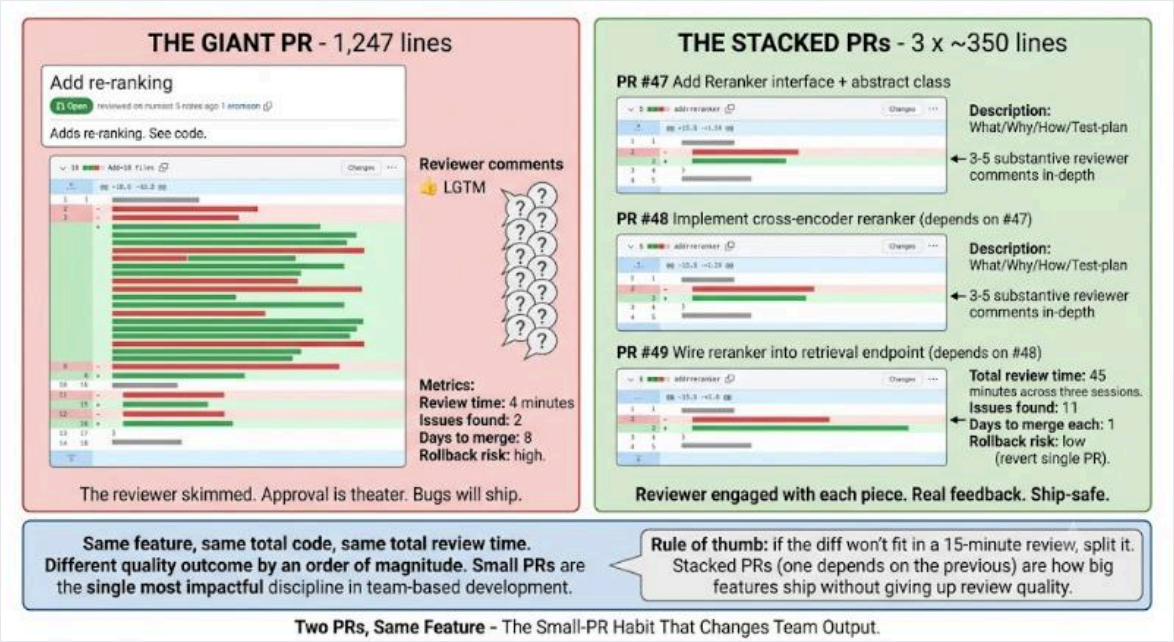
Markers: `<<<<<<< HEAD` (yours), `=======` (boundary), `>>>>>>> branch` (theirs). **Five-step ritual**: `git status` -> open + edit each file (or use editor's visual merge) -> `git add <file>` -> `git commit` (merge) or `git rebase --continue` (rebase).

Abort is safe: `git merge --abort` / `git rebase --abort` cleanly restores pre-conflict state. Watch for **semantic conflicts** Git can't detect - always run tests after merging.

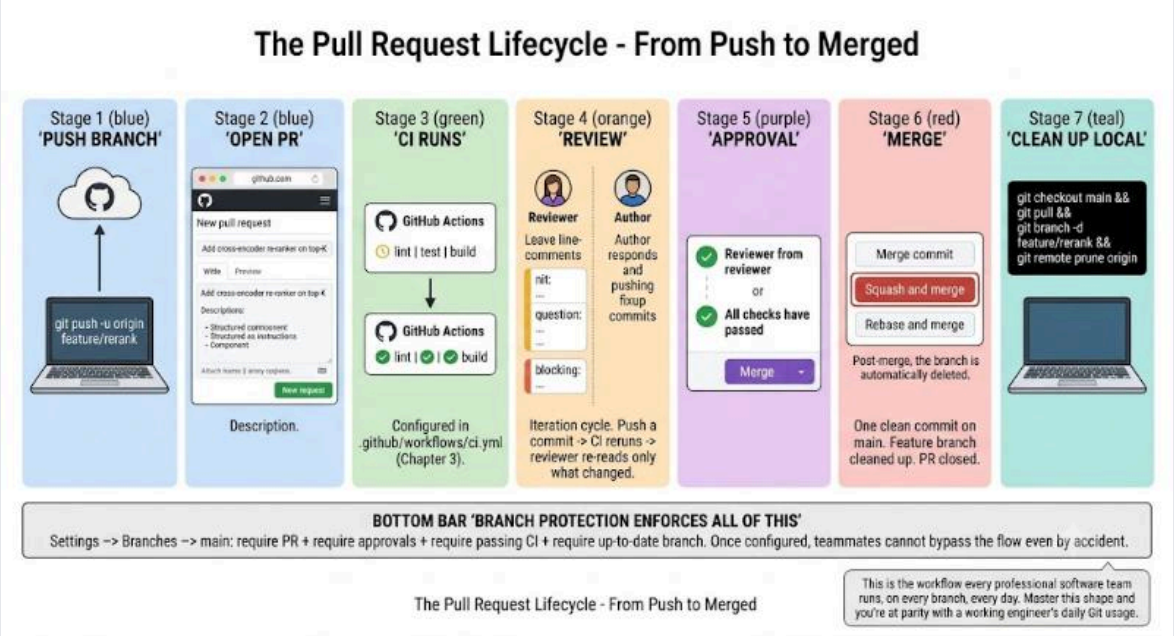
Prevent conflicts via small branches, frequent rebasing, and communication about big refactors.

PR ETIQUETTE AND CODE REVIEW - THE COLLABORATION SKILL

Claude • AI Assistant



Two PRs, Same Feature - The Small-PR Habit That Changes Team Output



The Pull Request Lifecycle - From Push to Merged

`markdown — A ready-to-use PR description template - drop into .github/pull_request_template.md`

```
<!--
  This template is auto-loaded when opening a PR because it lives at
  .github/pull_request_template.md in the repo.
-->

## What
<!-- 1-3 sentences describing what this PR changes. -->
```

```
## Why
<!-- What problem does this solve or opportunity does it unlock?
     Link the issue: Closes #123 (auto-closes on merge). -->


## How
<!-- For non-obvious implementations: approach, key decisions, alternatives.
-->


## Test plan
<!-- What tests were added? How can the reviewer verify locally?
     Include commands they should run. -->


## Screenshots / logs
<!-- Before/after for UI changes; sample output for CLI changes. -->


## Checklist
- [ ] Tests added and passing locally
- [ ] Docs / README updated if user-facing
- [ ] No leftover print / console.log / debug code
- [ ] No new dependencies without discussion
- [ ] Deploy notes below if any (migrations, env vars, feature flags)

## Deploy notes
<!-- Anything ops/deploy needs to know. Leave blank if none. -->
```

bash — PR-related commands and GitHub CLI for terminal workflows

```
# ---- Push branch and get a link to open the PR ----
git push -u origin feature/rerank
# GitHub prints: 'Create a pull request for feature/rerank by visiting:'
# https://github.com/priya-sharma/askmynotes/pull/new/feature/rerank
# Click that URL, fill in the form.

# ---- OR use the GitHub CLI (gh) to open PRs from terminal ----
# Install: brew install gh (macOS) / winget install github.cli (Windows)
gh auth login                          # one-time setup

# Open a PR straight from your feature branch
git checkout feature/rerank
```

```
gh pr create \
  --base main \
  --title "Add cross-encoder re-ranker on top-K retrieval" \
  --body-file .github/pull_request_template.md \
  --reviewer teammate-a,teammate-b \
  --label enhancement

# ---- Draft PR for early feedback ----
gh pr create --draft --title "WIP: adaptive chunk sizing"

# ---- Check status of open PRs ----
gh pr list
gh pr status                            # PRs relevant to you
gh pr view 47                           # view PR #47 details in terminal
gh pr checks                            # show CI status of current branch PR

# ---- Review a teammate's PR ----
gh pr checkout 47                       # switch to PR #47's branch locally
# ... run tests, poke around, add comments ...
gh pr review 47 --approve --body "Nice work. Confirmed MRR improvement
locally."
gh pr review 47 --request-changes --body "Please add tests for empty query
case"
gh pr review 47 --comment --body "nit: variable name could be clearer"

# ---- Merge from the CLI (rarely needed - usually merge via GitHub UI) ----
gh pr merge 47 --squash --delete-branch

# ---- Enable branch protection via API (or do it in GitHub UI: Settings ->
Branches) ----
# Recommended baseline for main:
#   - Require pull request before merging
#   - Require at least 1 approval
#   - Dismiss stale approvals when new commits are pushed
#   - Require status checks: CI must pass
#   - Require branches to be up to date before merging
#   - Do not allow bypassing (even for admins in strict setups)
```

Through **Pull Requests** (PRs on GitHub; Merge Requests on GitLab). A PR is a formal proposal: 'I want to merge this branch into main; please review the diff and approve.' A well-run PR ritual is the single biggest lever for code quality on a team - not the tooling, not the tests, but the human review conversation that PRs facilitate.

This subsection covers how to write a PR that reviewers will actually want to read, how to give and receive review feedback constructively, and how to avoid the common PR anti-

patterns.

WHAT A PR ACTUALLY IS

CONCEPT CARD

A PR is a **GitHub-side artifact** (not a Git concept) that links two branches: your **source branch** (e.g. `feature/rerank`) and a **target branch** (usually `main`).

GitHub computes the diff between the two, threads a discussion around it, runs any configured CI checks, and offers a Merge button once approvals and checks are green.

Creating one:

1. Push your feature branch: `git push -u origin feature/rerank`.
2. Open GitHub - it shows a yellow banner offering 'Compare & pull request' for recently pushed branches. Click it.
3. Fill in title and description.
4. Assign reviewers, labels, and (optionally) a milestone or project.
5. Submit.

GitHub then displays the PR page: the diff (per-file and unified), commits, comments, review state, CI check results, and the Merge button. This becomes the collaboration surface until merge.

THE PR SIZE RULE

⚠ WARNING

Yes - and the data is stark. Reviews on 1000-line PRs miss 90% of issues; reviews on 50-line PRs catch nearly all of them. Small PRs are the single most impactful discipline in team-based development.

Guideline: keep PRs under 400 lines of net change wherever possible. If a feature is bigger, split it into stacked PRs:

1. PR #1: 'Add re-ranker interface and abstract class' (200 lines - includes tests).
2. PR #2: 'Implement cross-encoder re-ranker' (150 lines - depends on #1 being merged).
3. PR #3: 'Wire re-ranker into retrieval endpoint' (100 lines - depends on #2).

Each PR is reviewable in 15 minutes. Total review time for the feature is the same as one giant PR, but review QUALITY is dramatically higher, and merges happen in days instead of weeks.

WRITING A GOOD PR DESCRIPTION

EXAMPLE CARD

The template that works everywhere:

markdown — Code Example

```markdown
## What
Adds a cross-encoder re-ranker on top of the vector retrieval step. Top-K
candidates from cosine similarity are re-scored by `ms-marco-MiniLM-L-6-v2`
and reordered before being passed to the LLM.

## Why
Retrieval was returning topically-relevant chunks but not always the most
answer-bearing ones. Re-ranking with a cross-encoder is the standard
next step in RAG pipelines. Closes #47.

## How
New module `retrieval/rerank.py` with a `Reranker` class. The retrieval
endpoint now calls `reranker.rerank(query, candidates)` after vector
search. Configurable via `RERANK_ENABLED` env var (default true).

## Test plan
- Added `tests/test_rerank.py` with 6 cases (empty, single, many, edge
cases)
- Manual eval: MRR on internal FAQ set went from 0.71 -> 0.78
- Local smoke test: `docker compose up && curl localhost:8000/ask?q=...`

## Checklist
- [x] Tests added and passing
- [x] README updated with `RERANK_ENABLED` env var
- [x] No leftover print/debug statements
- [ ] Deploy note: bumps container size by ~200MB (cross-encoder model)
```

Notice: the reviewer can approve, comment, or request changes just from reading the description - they don't have to reverse-engineer the intent from the diff. That respect for the reviewer's time is what gets your PRs approved quickly.

GIVING GOOD REVIEW FEEDBACK

CONCEPT CARD

Four rules:

1. Read the description first, then scan the diff, then dive into details. Big-picture first ('is this the right approach?'), then implementation ('is the code clean?'), then nits ('typo here').

2. Be specific and kind.

Instead of 'this is confusing', write 'I found this loop hard to follow because `i` is used for both the outer index and inside the helper - could `chunk_i` clarify?'. Reviewers who

explain WHY they're suggesting a change get better acceptance and teach the author something.

3. Distinguish blocking from non-blocking. Some teams use prefixes: `nit:` (small style issue - not blocking), `question:` (curious, not blocking), `blocking:` (must fix before merge). Reduces ambiguity around 'do I have to fix this or can we merge?'

4. Approve as soon as you're satisfied. Don't leave PRs hanging with 'looks good but' comments. If the change is worth making before merge, request it explicitly; otherwise, approve and let the author decide.

RECEIVING REVIEW FEEDBACK

CONCEPT CARD

Three modes:

- **Accept it silently** if the reviewer is right or the change is trivial. Fix, push, done.
- **Explain your reasoning** if you have a good reason for your choice. 'Good catch - I considered X but chose Y because Z. Happy to change if you feel strongly.' Most disagreements resolve here.
- **Escalate calmly** if you fundamentally disagree. 'I don't think we should block on this - let's chat 5 min offline?' Never let a review devolve into a heated comment thread. The written medium amplifies disagreement.

Never: take feedback personally, argue about style-preference points, or ignore comments without response. Every unanswered comment is a signal to the reviewer that you're not engaging.

THE 'DRAFT PR' PATTERN

CONCEPT CARD

Open a **Draft PR** (GitHub option when you create the PR - or convert an existing PR to draft). Draft PRs:

- Run CI and show the diff normally.
- Cannot be merged until marked 'Ready for review'.
- Signal to reviewers 'I'm not asking for approval yet; just early feedback'.

Common uses: (1) get architectural feedback on an approach before finishing all the details, (2) share work-in-progress so a teammate can review incrementally, (3) publish CI results (e.g. is my Docker build passing?) without formal approval load.

BRANCH PROTECTION RULES

CONCEPT CARD

Branch protection in GitHub (Settings -> Branches) lets you require:

- **PR before merge**: block direct pushes to main.
- **At least N approvals** before merge (typically 1-2).
- **Passing CI checks** before merge.
- **Up-to-date branch** (must rebase onto latest main before merging).
- **Signed commits** (for regulated environments).
- **Linear history** (blocks merge commits; forces rebase or squash).

Turn these on for `main` on any real project. They enforce the discipline you can't rely on humans to maintain manually.

MERGE STRATEGIES ON THE PR

CONCEPT CARD

Three options with different effects on `main`'s history:

- **Merge commit** (default): all feature branch commits + one merge commit land on main. Preserves full history but adds merge commits and noise.
- **Squash and merge**: all feature branch commits are combined into one commit on main. Clean linear history; each PR becomes one commit. This is what most modern teams pick.
- **Rebase and merge**: feature branch commits are replayed onto main individually. Linear history preserving per-commit granularity - good when your branch commits are already well-structured.

Recommendation: default to squash-merge unless your team has a specific reason otherwise. Main stays tidy; each PR is one revert-able unit.

ANALOGY

A Pull Request is like **submitting a paper for peer review**. The description is the abstract explaining the point.

The diff is the paper itself. Reviewers read the abstract to decide if the framing is right, skim the paper for major issues, then read carefully for details.

A tight abstract on a short paper gets thorough review; a rambling 200-page manuscript gets rubber-stamped. Reviewers are humans with limited attention - respect it and yours will be respected in return.

REAL WORLD APPLICATION

A team measured their PR quality by 'issues found per line reviewed'. On PRs under 200 lines, they found ~1 substantive issue per 100 lines.

On PRs over 1000 lines, they found ~1 issue per 700 lines - about 7x worse. When they enforced a 400-line soft limit, defect-escape rate dropped and merge-to-production time

halved.

Small PRs are not a stylistic preference; they measurably improve outcomes.

⚠ WARNING

Don't approve a PR you haven't actually read. 'LGTM 👍' on a 2000-line PR without meaningful engagement is worse than useless - it launders lack of review as approval, and everyone loses when the bug ships.

If you don't have time to review properly, say so and request another reviewer.

COMMON PITFALL / MISTAKE

Opening a PR titled 'WIP' or 'do not merge' and leaving it that way for weeks. Use **Draft PR** for genuinely-in-progress work. Draft PRs are visually distinct, cannot be accidentally merged, and don't clutter the reviewer's queue.

EXAM TIP CARD

Behavioral interview: 'Describe a code review you gave or received that changed your work.' Good answers demonstrate: (1) you take feedback constructively, (2) you can explain your reasoning when disagreeing, (3) you care about the code, not the ego. Naming a specific technical improvement that came from the review shows you actually engage.

PRACTICE QUIZ

Your PR has 850 lines of net change. A senior on your team pings you on Slack: 'can you split this?' Best response?

A) 'It's all one feature so it has to go together'

B) 'OK - let me identify natural break points and open 2-3 smaller stacked PRs'

C) 'Sure, let me squash to one commit so it looks smaller'

D) 'It's not that big - let's just merge'

▼ Reveal Correct Answer

Answer: B. Splitting into stacked PRs is the standard response and the senior is signaling a real quality concern. A is defensive. C misses the point - line count is what matters, not commit count. D is dismissive of a valid critique.

EXERCISE

On AskMyNotes: open a real PR against your own repo (yes, even solo). Write a description that follows the template (What / Why / How / Test plan / Checklist).

Enable branch protection on main requiring at least 1 review. Since you can't approve your own PR, add a friend as a collaborator or approve via a second GitHub account.

Merge via 'Squash and merge'. Now main has one clean commit representing the feature.

AI DEEP DIVE PROMPT

Ask an AI: '*Review this PR description and tell me what's missing or unclear. Then rewrite it following the What/Why/How/Test plan template. Finally, list three specific comments you would leave as a reviewer.*'

SUMMARY CARD

A **Pull Request** is a GitHub artifact linking a source branch to a target branch, with diff, discussion, CI results, and a Merge button. **Small PRs (< 400 lines) get real review**; giant PRs get rubber-stamped.

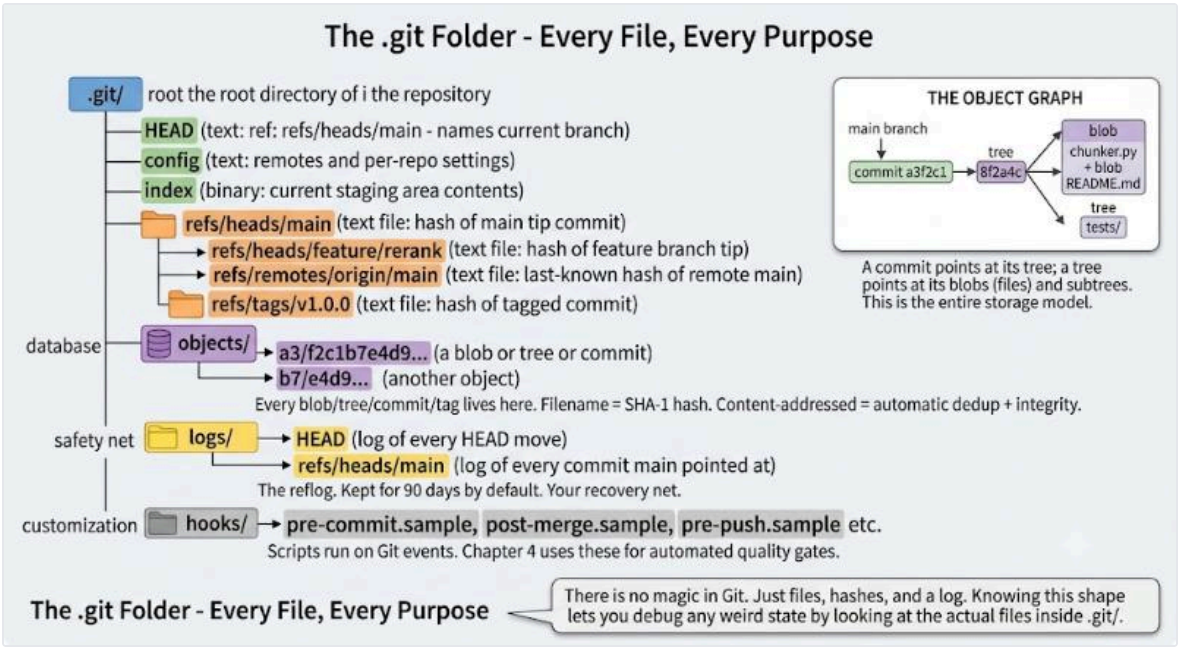
Structure descriptions with **What / Why / How / Test plan**. Give feedback that is **specific, kind, and distinguishes blocking from nitpick**.

Use **Draft PRs** for work-in-progress. Enforce quality with **branch protection**: require PR + approvals + passing CI.

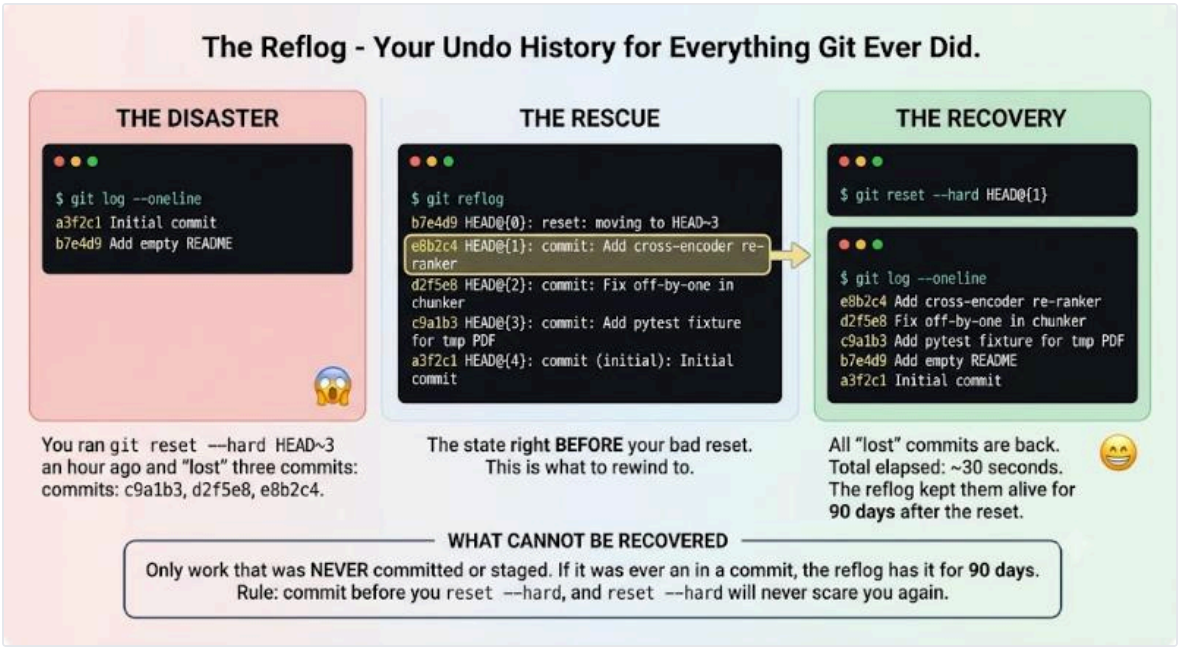
Prefer **squash-merge** for a clean linear main history. Every merged PR becomes one narratable unit in your project's story.

GIT INTERNALS AND RECOVERY - NOTHING IS UNRECOVERABLE

Claude • AI Assistant



The .git Folder - Every File, Every Purpose



The Reflog - Your Undo History for Everything Git Ever Did

bash — Recovery recipes - the five 'oh no' scenarios and their fixes

```
# ============================================================
# 1. Undo the LAST commit, keeping the changes to re-commit
# ============================================================
git reset --soft HEAD~1         # commit removed, staging area keeps changes
# edit / restage as needed:
git commit -m "Better message and split into separate commits"

# ============================================================
```

```
# 2. Undo a bad git reset --hard (get commits back)
# ============================================================
git reflog                        # find the pre-reset entry
# example output:
#   b7e4d9 HEAD@{0}: reset: moving to HEAD~3     <-- the disaster
#   e8b2c4 HEAD@{1}: commit: Add re-ranker       <-- the pre-disaster state
git reset --hard HEAD@{1}         # rewind to the good state

# ============================================================
# 3. Recover a deleted branch
# ============================================================
git reflog                         # find the branch's last commit hash
# or: git reflog show feature/rescued
git branch feature/rescued <hash-from-reflog>
git checkout feature/rescued

# ============================================================
# 4. Abort or undo a bad rebase
# ============================================================
# If the rebase is still in progress (conflict shown):
git rebase --abort                # cleanly restores pre-rebase state

# If you already completed the bad rebase:
git reflog                        # find the entry just BEFORE 'rebase
(start)'
git reset --hard HEAD@{N}

# ============================================================
# 5. Inspect any object by hash
# ============================================================
git show a3f2c1                   # commit or file-at-commit: human-readable
diff
git cat-file -t a3f2c1            # what TYPE of object? blob/tree/commit/tag
git cat-file -p a3f2c1            # raw pretty-printed object contents
git cat-file -s a3f2c1            # size of the object in bytes

# ============================================================
# BONUS: find truly unreachable objects (last resort)
# ============================================================
git fsck --lost-found             # lists commits not reachable by any ref
# Output includes hashes; inspect each with git show <hash>
# Rescue with: git branch rescue-<n> <hash>

# ============================================================
# DIAGNOSTIC: what's my current state?
# ============================================================
```

```
git status                          # working directory + staging + branch state
git branch -v                       # every local branch and its tip commit
git log --oneline --graph --all -20   # visual history across all branches
cat .git/HEAD                       # which ref (or hash) HEAD names
cat .git/refs/heads/main            # where main points
git reflog | head -20               # recent HEAD moves
git reflog show main | head -20     # recent main-branch moves
```

Student

I've been using Git commands as spells - I don't really know what happens inside `.git/` . And every time I do a `git reset --hard` , I'm terrified I'm about to lose work permanently. Can you demystify this?

Claude • AI Assistant

Yes - and this subsection is worth the extra care because knowing Git's internals gives you TWO superpowers: (1) nothing feels like magic anymore - every command has a mechanical explanation, and (2) **nothing is unrecoverable within about 30 days**. Git keeps every commit you ever made in a safety-net log called the **reflog**, even ones you 'deleted' or that were 'lost' during a bad rebase.

This subsection walks through the four object types Git stores, the ref system that points to them, the reflog, and the recovery moves that reverse the scariest 'oh no' moments.

THE .GIT FOLDER

Student

What's actually inside `.git/` ?

Claude • AI Assistant

CONCEPT CARD

Every Git repository has a hidden `.git/` directory at its root. Contents:

code — Code Example

```
.git/
  HEAD                # text file: 'ref: refs/heads/main' - names current
```

```
branch
  config                # per-repo Git config (remotes, user overrides)
  refs/
    heads/              # one file per LOCAL branch, each containing a
commit hash
      main              # e.g., contents: a3f2c1b7e4d9...
      feature/rerank    # e.g., contents: c9a1b3d2f5e8...
    remotes/
      origin/           # remote-tracking branches (updated by fetch/push)
    tags/               # one file per tag
  objects/              # THE database: every blob, tree, commit, tag lives
here
    a3/                 # objects are stored in subfolders by first 2 hash
chars
      f2c1b7e4d9...     # a compressed object file
    pack/               # older objects packed together for efficiency
  logs/                 # the reflog - history of every HEAD/branch move
    HEAD                # every place HEAD has ever pointed
    refs/heads/main     # every commit main has ever pointed at
  index                 # binary file: the current staging area
  hooks/                # optional scripts run on Git events (pre-commit,
post-merge, ...)
```

That's the entire structure. Text files pointing at hashes; a database of objects; a log of everything that ever happened. There's nothing mystical.

THE FOUR OBJECT TYPES

Student

Give me the object model.

Claude • AI Assistant

CONCEPT CARD

Git stores exactly four kinds of things in `.git/objects/` :

1. Blob - the raw content of a file. No filename, no permissions, just bytes. Identified by the SHA-1 hash of those bytes. If two files anywhere in history have the same content, they share exactly one blob (huge storage win).

2. Tree - a directory listing. A tree contains entries like `100644 blob a3f2c1... README.md` (mode + type + hash + name). Trees can reference blobs (files) or other trees

(subdirectories). Identified by the SHA-1 of the listing.

3. Commit - a snapshot.

A commit contains: the hash of the **root tree** (which recursively references every file's blob), the hash of the **parent commit** (or two parents for a merge), author name+email+timestamp, committer name+email+timestamp, and the commit message. Identified by the SHA-1 of all this.

4. Tag (annotated) - a named pointer to a commit, with a tagger and message. Different from a lightweight tag (which is just a ref file, no object).

Everything you see with `git log`, `git show`, `git diff` is one or more of these four object types. The whole system is a **content-addressed object database** with a few ref files on top pointing into it.

CONTENT ADDRESSING - WHY HASHES MATTER

Student

Why is everything hashed?

Claude • AI Assistant

CONCEPT CARD

Because **content addressing** gives you three properties for free:

Integrity: change one byte of a blob, tree, or commit -> different hash -> different object. Any tampering is trivially detectable. This is why Git is used in security-sensitive contexts.

Deduplication: identical content produces the same hash produces the same object stored exactly once. Copy a large file across 50 commits -> stored once. Rename a file with unchanged contents -> the blob is reused; only the tree changes.

Distributed synchronization: if two clones both have object `a3f2c1`, they have identical content. `git fetch` only needs to send objects the other side doesn't have. Efficiency without central coordination.

You can see the hashing yourself:

bash — Code Example

```
echo -n 'Hello' | git hash-object --stdin
# 5ab2f8a4323abafb10abb68657d9d39f1a775057
```

That's exactly the blob hash Git would assign if you committed a file containing 'Hello'.

HEAD AND REFS

Student

What's HEAD?

Claude • AI Assistant

CONCEPT CARD

HEAD is a text file at `.git/HEAD` that names 'where I am right now':

bash — Code Example

```
cat .git/HEAD
# ref: refs/heads/main          <-- symbolic ref: HEAD -> main branch
```

When you commit, Git:

1. Writes a new commit object.
2. Updates the branch that HEAD points to (e.g. `refs/heads/main`) to point at the new commit.

So `main` moves; HEAD keeps pointing at `main`; you're now 'on main at the new commit'.

When you `git checkout <branch>`, Git updates the HEAD file to say `ref: refs/heads/<branch>`, then rebuilds the working directory to match that branch's tip.

Detached HEAD state happens when HEAD points directly at a commit instead of a branch:

bash — Code Example

```
cat .git/HEAD
# a3f2c1b7e4d9...      <-- direct commit hash, not a symbolic ref
```

Common when you do `git checkout <commit-hash>` to inspect an old state. Commits made in detached HEAD are 'orphaned' - not on any branch - and Git warns you. Fix by creating a branch: `git checkout -b rescue-branch`.

THE REFLOG - YOUR SAFETY NET

Student

How does Git remember 'lost' commits?

Claude • AI Assistant

CONCEPT CARD

The **reflog** records every time HEAD (or a branch pointer) moves - commits, checkouts, resets, rebases, merges, everything. Each entry logs the previous position, the new position, and what caused the move. Kept for **~90 days** by default even for commits no branch points at.

bash — Code Example

```
git reflog
# c9a1b3d HEAD@{0}: commit: Add re-ranker tests
# b7e4d9a HEAD@{1}: checkout: moving from feature/rerank to main
# a3f2c1b HEAD@{2}: reset: moving to HEAD~2         <-- OH NO you reset
# d5e8f9c HEAD@{3}: commit: 'important work you thought was lost'
# ...
```

See that entry at `HEAD@{3}` ? That's a commit that no branch points at anymore (the reset erased it from `main`), but Git still knows about it in the reflog for ~90 days. To rescue it:

bash — Code Example

```
git branch rescue d5e8f9c        # create a branch pointing at the lost
commit
# or:
git reset --hard HEAD@{3}         # move current branch back to that state
```

This is the single most important recovery tool in Git. Nine out of ten 'help I destroyed my work' emergencies are solved by `git reflog` + `git reset --hard HEAD@{N}` or `git branch rescue <hash>` .

THE THREE LEVELS OF RESET

Student

What do `--soft` , `--mixed` , `--hard` do differently?

Claude · AI Assistant

CONCEPT CARD

`git reset` moves the current branch pointer somewhere else. The three modes control what happens to the staging area and working directory:

`git reset --soft <commit>`: branch moves; **staging area untouched**; working directory untouched. Effect: undo the commit but keep all its changes staged, ready to re-commit. Common use: 'I want to redo my last commit with a different message or split it into two'.

`git reset --mixed <commit>` (the default): branch moves; **staging area updated to match target commit**; working directory untouched. Effect: undo the commit AND unstage its changes, but keep the changes in your working directory.

Common use: 'I want to undo the commit and re-stage things differently'.

`git reset --hard <commit>`: branch moves; **staging area matches target**; **working directory matches target - your uncommitted edits are DISCARDED**. Effect: everything reverts to the target commit. Common use: 'I want to totally undo everything since <commit>'.

`--hard` is the one that loses uncommitted work. The reflog can rescue you from a wrong `--hard` on a committed state, but it cannot rescue uncommitted edits that were never in a commit or the reflog. Rule: **commit before you reset --hard**.

COMMON RECOVERY SCENARIOS

Student

Give me the recipe book.

Claude · AI Assistant

EXAMPLE CARD

The five 'oh no' moments and how to fix each:

1. 'I accidentally committed something I shouldn't have (and haven't pushed).'

bash — Code Example

```
git reset --soft HEAD~1    # undo commit, keep changes staged
# edit / unstage as needed, then re-commit
```

2. 'I did `git reset --hard` and want my commits back.'

```bash
git reflog                  # find the entry BEFORE the reset
git reset --hard HEAD@{2}   # jump back to that state
```

3. 'I deleted a branch by mistake.'

```bash
git reflog                  # find the branch's last commit hash
git branch feature/rescued <hash>
```

4. 'A rebase went wrong and I want the pre-rebase state.'

```bash
git rebase --abort          # if the rebase is still in progress
# or, if you already completed the bad rebase:
git reflog
git reset --hard HEAD@{N}   # to the commit right before rebase started
```

5. 'I want to see a commit's contents by hash.'

```bash
git show a3f2c1              # commit metadata + diff
git cat-file -p a3f2c1       # raw commit object contents
git cat-file -t a3f2c1       # what type of object is this?
(blob/tree/commit/tag)
```

WHAT CANNOT BE RECOVERED

Student

What actually is lost forever?

Claude • AI Assistant

> ⚠ **WARNING**
>
> Only three categories are unrecoverable:

1. **Uncommitted work never in the reflog** - working-directory edits you never committed AND never staged when you ran `git reset --hard` or `git clean -fdx`. If it wasn't in a commit or the reflog, Git doesn't know it existed.

1. **Files never added to Git** - if you never ran `git add`, Git has no history of that file's contents. Untracked files removed by `git clean` are gone.

1. **Objects older than the reflog expiration** - default 90 days for reachable objects, 30 days for unreachable ones. `git gc` (garbage collection) permanently removes expired unreachable objects. Nothing you're likely to encounter as a student.

For everything else - deleted branches, botched rebases, rewritten history, force-pushed remotes - the reflog is your friend and recovery is fast.

TAGS

Student

One more thing - what are tags?

Claude • AI Assistant

CONCEPT CARD

Tags are named pointers to specific commits, usually for releases:

bash — Code Example

```
git tag v1.0.0                                  # lightweight tag
git tag -a v1.0.0 -m "First stable release"     # annotated tag (recommended
for releases)
git push origin v1.0.0                           # push tag to remote
git push origin --tags                           # push all tags
```

Unlike branches, tags **don't move**. Once you tag `v1.0.0`, that name always points to the same commit. Convention: use **semantic versioning** for tags: `v<MAJOR>.<MINOR>.<PATCH>`. Chapter 4's CD flow triggers deploys on tag pushes.

ANALOGY

Git's object database is like a **library where every book is filed by its content-hash instead of a shelf number**. Two identical books get one copy on the shelf but many library cards.

HEAD is your bookmark - it names the book you're currently reading. Branches are labeled tabs on shelves.

Tags are permanent nameplates. The reflog is the library's checkout log - every book you've ever pulled, kept for 90 days, findable even if you forget which shelf you left something on.

REAL WORLD APPLICATION

A senior engineer at a bank once demonstrated the reflog to a room of terrified junior devs by force-pushing to overwrite three days of a colleague's committed work, then recovering all of it in 30 seconds via `git reflog` + `git branch rescue` . The moral of the story wasn't 'force-push freely'; it was that fear of Git accidents is almost entirely misplaced once you know the reflog exists.

Every senior engineer has this recovery move memorized.

WARNING

`git gc --prune=now` and `git reflog expire --expire=now --all` permanently remove unreachable objects and reflog entries. Do NOT run these to 'clean up' unless you're absolutely sure - they defeat the recovery mechanisms this subsection depends on.

Git's automatic GC is conservative and safe; leave it alone.

COMMON PITFALL / MISTAKE

Assuming that a branch delete or a force-push permanently removes commits. It doesn't - the commits remain in the object database until GC expires them (30-90 days).

Reachable via `git reflog` and `git fsck --lost-found` . This is why 'accidentally deleted the branch' is a five-second fix, not a career-ending crisis.

EXAM TIP CARD

Star question: **'You did `git reset --hard HEAD~3` an hour ago. Can you get the 3 commits back?

How?'** Perfect answer: 'Yes. `git reflog` lists the commit HEAD pointed at before the reset - probably at `HEAD@{1}` .

Then `git reset --hard HEAD@{1}` restores the branch to that state. Alternatively `git branch rescue-branch HEAD@{1}` to create a new branch there.

The reflog keeps entries for 90 days by default.'

PRACTICE QUIZ

You force-pushed to feature/rerank, overwriting a teammate's two commits. Their commits are 'gone' from the remote. Can they recover?

A) No - force-push is destructive and permanent

B) Yes - if they still have the commits in their LOCAL reflog, they can push them back

C) Only if they had opened a PR containing them

D) Only within 60 seconds of the force push

▼ Reveal Correct Answer

Answer: B. Their local reflog still has the commits. They can `git reset --hard HEAD@{N}` to restore their branch locally, then push again (potentially with --force-with-lease). This is why --force-with-lease matters and why the reflog is always your friend.

EXERCISE

On AskMyNotes (in a scratch branch, please): (a) commit something, (b) `git reset --hard HEAD~1` to 'delete' it, (c) run `git reflog` and identify the commit hash you just erased, (d) run `git branch rescued <hash>` to bring it back as a new branch, (e) `git log --oneline rescued` - confirm your 'lost' commit is there. Now you'll never fear reset again.

AI DEEP DIVE PROMPT

Ask an AI: '*Explain Git's four object types (blob, tree, commit, tag) with a concrete example: what gets stored when I commit one Python file. Then explain the reflog and walk me through recovering from an accidental git reset --hard.*'

SUMMARY CARD

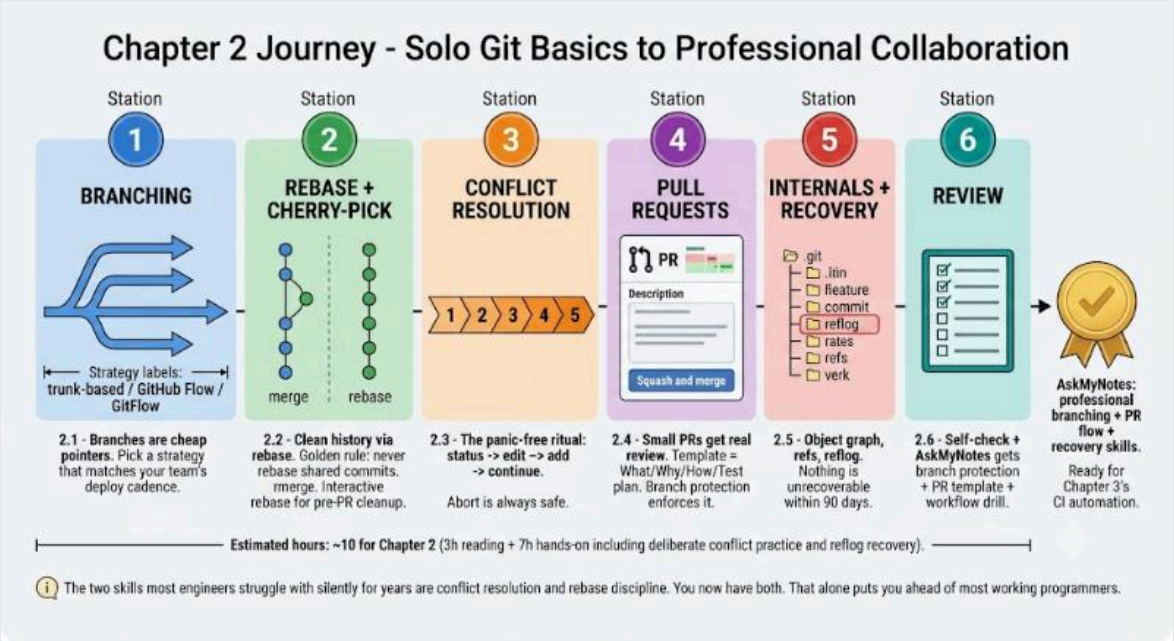
Git stores everything as four object types in `.git/objects/` : **blob** (file bytes), **tree** (directory listing), **commit** (snapshot: tree hash + parent hash + message), **tag** (annotated pointer). All content-addressed by SHA hash -> automatic integrity, deduplication, distributed sync.

HEAD is a text file naming the current branch; branches are text files naming a commit hash. The **reflog** records every HEAD/branch movement for ~90 days - your safety net for lost commits after reset/rebase/branch-delete.

Recovery mantra: `git reflog` -> find the pre-accident entry -> `git reset --hard HEAD@{N}` or `git branch rescue <hash>` . Only truly-uncommitted, never-staged work is unrecoverable.

CHAPTER 2 REVIEW & SUMMARY

Claude · AI Assistant



Chapter 2 Journey - Solo Git Basics to Professional Collaboration

bash — Chapter 2 command reference - the branching, rebasing, PR-flow, recovery kit

```
# ---- Branching (2.1) ----
git checkout -b feature/x                    # create + switch
git switch -c feature/x                      # modern equivalent
git branch                                   # list local branches
git branch -a                                # list all (local + remote)
git branch -d feature/x                      # delete merged branch
git push origin --delete feature/x           # delete remote branch

# ---- Rebase and cherry-pick (2.2) ----
git rebase main                              # replay current branch on main
git rebase -i HEAD~5                         # interactive rebase last 5
commits
git rebase --continue / --abort              # after conflicts / to bail
git pull --rebase                            # cleaner than plain git pull
git cherry-pick <hash>                       # copy specific commit here
git push --force-with-lease origin feature/x  # safer force-push after rebase

# ---- Conflict resolution (2.3) ----
git status                                   # what needs resolving?
# edit file, remove <<<<<<< ======= >>>>>>> markers
git add <file>
git commit                                   # for merge
git rebase --continue                        # for rebase
git merge --abort / git rebase --abort       # escape hatch
```

```
git checkout --ours <file>                    # accept your side wholesale
git checkout --theirs <file>                  # accept their side wholesale

# ---- PR flow (2.4) ----
git push -u origin feature/x                  # first push sets upstream
gh pr create --base main --title "..."        # open PR from terminal
gh pr list / view / checks / merge            # PR ops via GitHub CLI
# Then in GitHub UI: request reviewers, wait for CI + approvals,
# choose 'Squash and merge', delete branch.

# ---- Internals + recovery (2.5) ----
git reflog                                    # every HEAD move (recovery
lifeline)
git reset --soft HEAD~1                       # undo commit, keep changes
staged
git reset --mixed HEAD~1                      # undo commit, unstage changes
(default)
git reset --hard HEAD@{2}                     # jump to a reflog entry
(recovery)
git branch rescue <hash>                      # revive a 'lost' commit as a
branch
git show <hash>                               # inspect any commit/blob/tree
git cat-file -p <hash>                        # raw contents of any object
git fsck --lost-found                         # find fully-unreachable
commits
git tag -a v1.0.0 -m "First release"          # annotated tag (releases)
git push origin --tags                        # publish tags
```

CHAPTER SUMMARY

Chapter 2 took Git from single-developer basics into professional collaboration. **Section 2.1** covered branching - what a branch actually is (a movable pointer to a commit), the universal feature-branch workflow, and the three real-world strategies (trunk-based, GitHub Flow, GitFlow) so you can answer 'what's your branching model?' in any interview.

Section 2.2 went deep on rebasing and cherry-picking - the difference between merge and rebase, the golden rule ('never rebase commits others have pulled'), interactive rebase for cleaning up local history, and safe force-pushing with `--force-with-lease`. **Section 2.3** unlearned the panic reaction to merge conflicts - conflict-marker anatomy, the five-step resolution ritual, editor tooling, the abort escape hatch, and the important distinction between textual and semantic conflicts.

Section 2.4 covered PR etiquette - why small PRs (< 400 lines) get real review, the What/Why/How/Test-plan template, giving and receiving feedback constructively, draft PRs for WIP, branch protection, and merge strategies. **Section 2.5** demystified Git internals - the

four object types (blob, tree, commit, tag), content addressing, HEAD and refs, and most importantly the **reflog** as the recovery safety net that makes nearly every 'oh no' moment fixable in seconds.

THE ONE-PAGE MENTAL MODEL

Git is a content-addressed object database (`.git/objects/`) with a few text-file pointers on top (`.git/refs/heads/`). Branches are cheap pointers; PRs are the collaboration ritual around merging them.

Rebase produces clean linear history but rewrites commits - safe on your own local work, catastrophic on shared history. Conflicts are decisions Git can't make alone - resolve calmly with the five-step ritual.

And when things go wrong, the **reflog** remembers everything for 90 days - nothing you can do to your local repo is truly unrecoverable.

WHAT'S NEXT

Chapter 3 takes the human-driven quality of Chapter 2 (branches, PRs, review) and adds the **automated** side: testing and continuous integration. You'll level up pytest with fixtures, parametrize, and mocking; understand the test pyramid and coverage; then wire everything into **GitHub Actions** so tests run automatically on every push and PR.

By end of Chapter 3, your AskMyNotes repo has a green CI badge and no code merges to main without passing tests.

SELF-CHECK

Before moving on, verify you can:

1. Explain what a branch is at the file level (a text file in `.git/refs/heads/` containing a hash).
2. Describe trunk-based, GitHub Flow, and GitFlow with one advantage and one disadvantage of each.
3. State the golden rule of rebasing and give one concrete example of what it prevents.
4. Resolve a merge conflict end to end (understand markers, edit, `git add` , continue).
5. Explain why `git rebase --abort` is safe and why `git merge --abort` is safe.
6. Write a PR description following the What/Why/How/Test-plan template.
7. Recover from `git reset --hard HEAD~3` using the reflog.
8. Name Git's four object types and what each stores.

If any of these are shaky, re-read the relevant subsection. Chapter 3 assumes all eight.

CHAPTER 2 EXERCISE - ASKMYNOTES GETS A PROFESSIONAL WORKFLOW

Do this before starting Chapter 3:

1. Enable branch protection on `main`: require PR, at least 1 approval (or 0 if solo), require passing status checks (none configured yet - Chapter 3 adds them).
2. Add `.github/pull_request_template.md` with the What/Why/How/Test-plan template.
3. Create a feature branch `feature/error-handling`, make 4 messy commits.
4. Interactive-rebase them into 2 clean commits (`git rebase -i HEAD~4`).
5. Push, open a PR with a proper description, self-review, merge via squash-and-merge, delete the branch.
6. Deliberately reset `main` to 2 commits ago (`git reset --hard HEAD~2`), then use the reflog to recover.
7. Screenshot the recovered log for your notes.

SUMMARY CARD

Chapter 2 turned solo Git into professional collaboration: **branches** (cheap pointers) and strategies (trunk-based / GitHub Flow / GitFlow); **rebase vs merge** with the golden rule and interactive rebase for tidiness; **merge conflicts** as calm mechanical work via the five-step ritual with `--abort` as a safe escape; **Pull Requests** with small size discipline, structured descriptions, constructive review, and branch protection; **Git internals** (four object types, HEAD, refs, reflog) demystifying the tool and enabling recovery from nearly any accident. AskMyNotes now has a professional workflow ready for Chapter 3's automated testing and CI.